

目标检测

导航

1	前言	4
2	准备知识	4
2.1	Loss 合辑	4
2.1.1	Focal loss	4
2.1.2	Distribution Focal Loss	5
2.1.3	Quality Focal Loss	5
2.1.4	Generalized Focal Loss	5
2.2	IoU 及其变体	5
2.2.1	IOU	5
2.2.2	GIoU	6
2.2.3	DIoU	6
2.2.4	CIoU	7
2.2.5	EIoU	7
2.2.6	SIoU	8
2.3	NMS 合辑	8
2.3.1	NMS	8
2.3.2	多类别 NMS	9
2.3.3	Soft-NMS	9
2.3.4	Softer-NMS	9
2.3.5	Fast NMS	10
2.3.6	Weighted-Boxes-Fusion, WBF	11
3	RCNN 系列	12
3.1	RCNN	12
3.2	Fast RCNN	12
3.3	Faster RCNN	12
3.3.1	Backbone	12
3.3.2	Head	12
3.3.3	Region Proposal Networks	14

3.3.4	RoI Pooling	15
3.3.5	Classification	16
3.3.6	推理阶段	17
3.4	Mask RCNN	17
4	YOLO 系列	17
4.1	YOLOV1	17
4.2	YOLOV2	17
4.3	YOLOV3	17
4.3.1	Darknet53	19
4.3.2	Neck	19
4.3.3	Head	19
4.3.4	推理阶段	20
4.3.5	后处理	22
4.4	YOLOV4	23
4.4.1	CSPDarknet53	24
4.4.2	SPP 模块	24
4.4.3	Feature pyramid network	24
4.4.4	Path Aggregation Network	25
4.4.5	稀疏化训练	26
4.5	YOLOV5	26
4.5.1	图像加权采样	26
4.5.2	推理阶段	27
4.6	YOLOX	29
4.6.1	Strong augmentation	29
4.6.2	Yolox-Darknet53	30
4.6.3	Neck	30
4.6.4	Prediction 层	30
4.7	YOLOV6	33
4.8	YOLOV7	34
4.8.1	ELAN	34
4.8.2	TransitionBlock&MP	35
4.8.3	Neck	35
4.8.4	Head	36
4.8.5	正负样本筛选	36
4.9	YOLOV8	36
4.9.1	Backbone & Neck	38
4.9.2	Head	38
4.9.3	TaskAlignedAssigner	39

- 4.9.4 Loss 计算 40
- 4.10 YOLOV9 40
 - 4.10.1 创新点 40
 - 4.10.2 信息瓶颈原理 40
 - 4.10.3 可逆函数 41
 - 4.10.4 对轻型模型的影响 41
 - 4.10.5 可编程梯度信息 (PGI) 41
 - 4.10.6 通用高效层聚合网络 (GELAN) 41
- 5 Transformer 系列 42
 - 5.1 DETR 42
 - 5.1.1 目标函数 43
 - 5.1.2 Object queries 44
 - 5.1.3 DETR 的局限性 44
- 6 半监督学习 44
 - 6.1 Softer Teacher[2] 44

1 前言

以下是在学习基于深度学习方法实现目标检测的过程中，总结的论文、学习报告、博客等等，以及针对部分问题提出的个人看法，特此记录！

2 准备知识

2.1 Loss 合辑

2.1.1 Focal loss

用于处理目标检测中正负样本比例失衡的问题。【一张图像可以有成千上万个候选框，但是只有很少的一部分包含目标】**【也可以处理 long-tail 问题】**

以二分类为例：

$$CE = -y \log y' - (1 - y) \log(1 - y') = \begin{cases} -\log y', & y = 1 \\ -\log(1 - y'), & y = 0 \end{cases} \quad (1)$$

其中， $y' \in (0,1)$ 表示输出类别对应的概率。

通常会在交叉熵的前面加一个参数 α 控制正负样本对 loss 的贡献，即

$$CE = \begin{cases} -\alpha \log y', & y = 1 \\ -(1 - \alpha) \log(1 - y'), & y = 0 \end{cases} \quad (2)$$

此外，**【当某类样本的概率越大，也就也意味着更容易被识别为该样本】**。改进式 (2)：

$$FL = \begin{cases} -\alpha(1 - y')^\gamma \log y', & y = 1 \\ -(1 - \alpha)y'^\gamma \log(1 - y'), & y = 0 \end{cases} \quad (3)$$

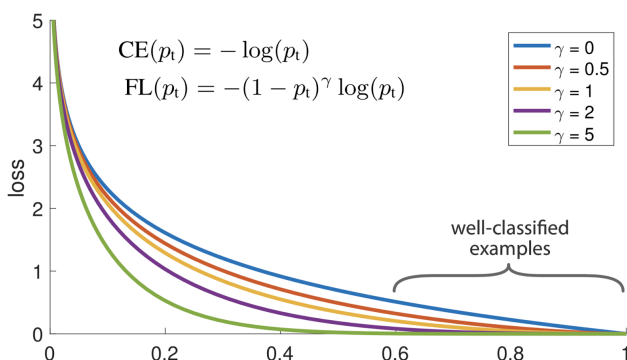


图 1: 正样本概率

由图1不难发现 **【仔细看图】**: y' 越大, FL 越小, 它对总体的 FL 的贡献就越小。**【此时标签为 1】【降低高置信度样本的损失, 通俗地讲就是, 容易被识别的类别就只要求预测概率超过 0.5 就可以了, 不用过度追求预测概率为 1, 应该更加关注难以分类的样本】**

```
def forward(self, pred, true):
    loss = self.loss_fcn(pred, true)
    pred_prob = torch.sigmoid(pred) # prob from logits
    p_t = true * pred_prob + (1 - true) * (1 - pred_prob)
    alpha_factor = true * self.alpha + (1 - true) * (1 - self.alpha)
    modulating_factor = (1.0 - p_t) ** self.gamma
    loss *= alpha_factor * modulating_factor
    if self.reduction == 'mean': return loss.mean()
    elif self.reduction == 'sum': return loss.sum()
    else: return loss
```

2.1.2 Distribution Focal Loss

$$\mathbf{DFL}(S_i, S_{i+1}) = -((y_{i+1} - y) \log(S_i) + (y - y_i) \log(S_{i+1})) \quad (4)$$

其中, $S_i = \frac{y_{i+1}-y}{y_{i+1}-y_i}$, $S_{i+1} = \frac{y-y_i}{y_{i+1}-y_i}$, $\hat{y} = \sum_{j=0}^n P(y_j) y_j = S_i y_i + S_{i+1} y_{i+1} = \frac{y_{i+1}-y}{y_{i+1}-y_i} y_i + \frac{y-y_i}{y_{i+1}-y_i} y_{i+1} = y$

将标签转换为 DFL 的具体过程为:

- 将标签值 xywh 转换为 ltrb
- 计算 ltrb 四个值, 转换为 DFL 所需的积分形式 (预测值 + 临近预测值)

$$y = \text{中心距某条边的距离/当前下采样倍数} \quad (5)$$

比如目标中心点距上边 73 px, 在最大下采样 32 倍的特征图中, 计算其在 DFL 中的映射值 $y = 73/32 = 2.3$ 。取整后得到的预测值为 $y_i = 2$, 临近值 $y_{i+1} = 3$ 。根据映射值与预测值、临近预测值的接近程度, 计算各自的权重为 $y - y_i = 0.3$ 和 $y_{i+1} - y = 0.7$

2.1.3 Quality Focal Loss

$$\mathbf{QFL}(\sigma) = -|y - \sigma|^\beta ((1 - y) \log(1 - \sigma) + y \log(\sigma)) \quad (6)$$

2.1.4 Generalized Focal Loss

模型去估计两个量 y_l 和 y_r , ($y_l < y_r$) 对应的 p_{y_l} 和 p_{y_r} , ($p_{y_l} \geq 0, p_{y_r} \geq 0, p_{y_l} + p_{y_r} = 1, \hat{y} = y_l p_{y_l} + y_r p_{y_r}$)

$$\mathbf{GFL}(p_{y_l}, p_{y_r}) = -|y - (y_l p_{y_l} + y_r p_{y_r})|^\beta ((y_r - y) \log(p_{y_l}) + (y - y_l) \log(p_{y_r})) \quad (7)$$

2.2 IoU 及其变体

2.2.1 IOU

$$IoU = \frac{|A \cap B|}{|A \cup B|}, IoU \in [0, 1] \quad (8)$$

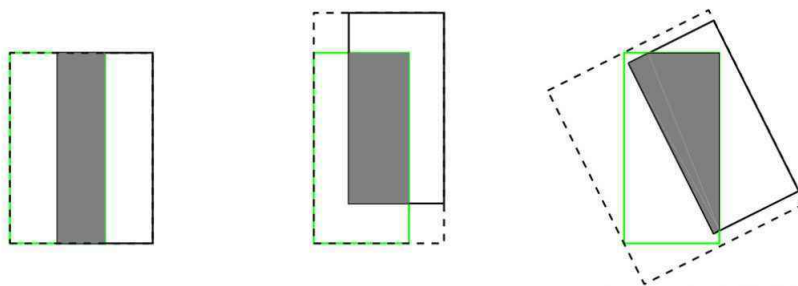


图 2: 相同的 IoU 对应不同的重叠方式。绿色方框为真实框；灰色区域为重叠区域；黑色方框为预测框

提醒:

- 可以反映预测框和真框的相似度，满足尺度不变性【对尺度不敏感】；
- 如果两个框没有相交【IoU=0】，影响梯度回传；
- 无法精确的反映两个框的重合度大小图2

2.2.2 GIoU

$$GIoU = IoU - \frac{|C - (A \cup B)|}{|C|}, GIoU \in [-1, 1] \quad (9)$$

其中， $|C|$ 为预测框和真实框的最小闭包区域面积； $|C - (A \cup B)|$ 为不属于两个框的区域。当且仅当真实框和预测框重合时， $GIoU = 1$ ；不存在交集时， $GIoU = -1$ 。

2.2.3 DIoU

$$DIoU = IoU - \frac{\rho^2(b, b^{gt})}{c^2} \quad (10)$$

其中， b, b^{gt} 分别为预测框和真实框的中心点； ρ 为欧式距离函数； c 为预测框和真实框的最小闭包区域的对角线距离。

提醒:

- 和 GIoU 一样，当预测框和真实框不存在重叠时，仍然可以正常梯度下降；
- 可以直接衡量预测框和真实框的距离；
- 当预测框和真实框为包含状态时，GIoU 退化为 IoU，DIoU 仍有可能继续更新梯度；
- DIoU 也可以应用于 NMS 中。

2.2.4 CIoU

$$CIoU = IoU - \frac{\rho^2(b, b^{gt})}{c^2} - \alpha v \quad (11)$$

$$v = \frac{4}{\pi^2} \left(\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h} \right)^2 \quad (12)$$

$$\alpha = \begin{cases} 0, & \text{if } IoU < 0.5 \\ \frac{v}{1-IoU+v}, & \text{if } IoU \geq 0.5 \end{cases} \quad (13)$$

提醒:

- 当预测框和真实框重叠较少时，即 $IoU < 0.5$ ，CIoU 退化为 DIoU；
- 当 $IoU \geq 0.5$ 时，预测框和真实框的中心点存在重合的可能性，因此需要引入纵横比进一步调整预测框，依据于：
- 当预测框和真实框的纵横比不一致时， v 会比较大，可以继续保持 loss 的梯度。

2.2.5 EIoU

$$EIoU = IoU - \frac{\rho^2(b, b^{gt})}{c^2} - \frac{\rho^2(w, w^{gt})}{C_w^2} - \frac{\rho^2(h, h^{gt})}{C_h^2} \quad (14)$$

其中， C_w 和 C_h 分别为覆盖预测框和真实框的最小外接框的宽度和高度。

提醒:

- CIoU 无法很好的度量预测框的宽和高具体的回归程度；
- EIoU 还结合了 Focal loss $【L_{EIoU} = IoU^\gamma(1 - EIoU)】$ ，来降低与目标重叠较少的大量锚对回归框损失的贡献。

```
.....
iou = inter / union
if CIoU or DIoU or GIoU:
    cw = torch.max(b1_x2, b2_x2) - torch.min(b1_x1, b2_x1) # 最小外接框的宽
    ch = torch.max(b1_y2, b2_y2) - torch.min(b1_y1, b2_y1) # 最小外接框的高
    if CIoU or DIoU:
        c2 = cw**2 + ch**2 + eps
        rho2 = ((b2_x1 + b2_x2 - b1_x1 - b1_x2)**2 + (b2_y1 + b2_y2 - b1_y1 - b1_y2) ** 2)/4
        if CIoU:
            v = (4/math.pi**2) * torch.pow(torch.atan(w2/(h2 + eps)) - torch.atan(w1/(h1 + eps)), 2)
            with torch.no_grad(): alpha = v / (v - iou + (1 + eps))
            return iou - (rho2 / c2 + v * alpha)
        return iou - rho2 / c2
    c_area = cw * ch + eps
    return iou - (c_area - union) / c_area
return iou
```

α -IoU

$$\begin{aligned} L_{\alpha-IoU} &= 1 - IoU^\alpha \\ L_{\alpha-GIoU} &= 1 - IoU^\alpha + \left(\frac{|C - (A \cup B)|}{|C|} \right)^\alpha \\ &\dots \end{aligned} \quad (15)$$

提醒:

- $\alpha = 3$ 时, 增加了 high IoU 目标的损失和梯度, 从而提高了回归框的精度【直观来看, 添加幂次运算, 可以放大损失的敏感度】【测试了一下, 没啥用】;
- 不会引入额外的参数, 也不会增加训练或推理的时间。

2.2.6 SIoU

【Submitted on 25 May 2022】额外考虑了回归框与真实框之间的向量角度

动机: 迄今为止, 提出的方法都没有考虑到真实框与预测框之间的匹配方向, 进而导致训练收敛速度慢, 预测框在训练的过程中“四处游荡”。

具体请参考[AI 视觉网奇](#)

2.3 NMS 合辑

动机: 在执行目标检测任务时, 可能对同一目标有多次检测, 因此有必要“清理检测”, 使得每个对象只得到一个检测。

2.3.1 NMS

非极大值抑制 (Non-Maximum Suppressin, NMS) 局部最大搜索

Algorithm 1: 非极大值抑制

输入: 候选框集合 $\mathcal{B}$ 及其对应的置信度 $\mathcal{S}$; 阈值 N_t 【常用 0.3-0.5】

最终检测结果 $\mathcal{D} \leftarrow \{\}$

while $\mathcal{B} \neq \text{empty}$ **do**

 步骤一, 从 $\mathcal{B}$ 中选择最大 score 的检测框 m ;

 步骤二, 将其从 $\mathcal{B}$ 中移除并加入到最终检测结果 $\mathcal{D}$ 中;

 步骤三, 将 $\mathcal{B}$ 中剩余检测框 b_i 与 m 的 IOU 大于 N_t 的框从 $\mathcal{B}$ 中移除;

end

输出: 最终检测结果

提醒：

- 在使用 NMS 之前需要预选框预处理【NMS 耗时】；
- 将相邻检测框的分数强制归零，当一个真实物体在重叠区域出现时，会致使该目标检测失败，降低平均检测率；
- 阈值选取过于主观；
- 只能 CPU 计算

2.3.2 多类别 NMS**提醒：**

- 1的做法是把所有 boxes 放在一起做 NMS，没有考虑类别【某一类的 boxes 不应该因为它与另一类最大得分 boxes 的 iou 值超过阈值而被筛掉】
- 实现方式：为每个类别单独做 NMS【给 box 坐标添加一个偏移量，偏移量由类别索引决定】

2.3.3 Soft-NMS

不像 NMS 一样粗暴的删除 IOU 大于阈值的检测框，Soft-NMS 衰减分数。区别在于步骤三，前者：

$$s_i = \begin{cases} s_i, & iou(m, b_i) < N_t \\ 0, & iou(m, b_i) \geq N_t \end{cases} \quad (16)$$

后者有两种实现形式：

- 线性加权

$$s_i = \begin{cases} s_i, & iou(m, b_i) < N_t \\ s_i (1 - iou(m, b_i)), & iou(m, b_i) \geq N_t \end{cases} \quad (17)$$

- 高斯加权

$$s_i = s_i e^{-\frac{iou(m, b_i)^2}{\sigma}}, \forall b_i \notin \mathcal{D} \quad (18)$$

提醒：

- 不需要重新训练原有的模型，二值化时等价于 NMS；
- $\mathcal{B}$ 为空后，所有的候选框都还在，只是置信度发生了变化，还需要阈值处理一次。

2.3.4 Softer-NMS

动机： NMS 仅使用了置信度得分，没法反映出候选框的定位精确度【分类置信度和定位置置信度非正相关】
Softer-NMS 引入了两个假设：

- 假设 1: 候选框的定位信度 (x_e) 是高斯分布;

$$P_{\Theta}(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-x_e)^2}{2\sigma^2}} \quad (19)$$

- 假设 2: 真实框的定位信度 (x_g) 是狄拉克分布, 即标准方差为 0 的高斯分布极限

$$P_D(x) = \delta(x - x_g) \quad (20)$$

用 KL 散度来衡量两个分布的相似度:

$$L_{reg} = \arg \min_{\Theta} D_{KL}(P_D(x) || P_{\Theta}(x)) \quad (21)$$

$$= \int P_D(x) \log P_D(x) dx - \int P_D(x) \log P_{\Theta} dx \quad (22)$$

$$= -\log P_{\Theta}(x_g) + C_1 \quad (23)$$

$$= \frac{(x_g - x_e)^2}{2\sigma^2} + \frac{\log \sigma^2}{2} + C_2 \quad (24)$$

其中 C_1, C_2 均为常量。根据链式法则分别对 x_e 和 σ 求偏导:

$$\frac{dL_{reg}}{dx_e} = \frac{x_e - x_g}{\sigma^2} \quad (25)$$

$$\frac{dL_{reg}}{d\sigma} = -\frac{(x_e - x_g)^2}{\sigma^3} + \frac{1}{\sigma} \quad (26)$$

设定 $\alpha = \log(\sigma^2)$ 【动机和4.3节中回归框长宽缩放到对数空间一样】, 此时

$$L_{reg} \propto \frac{e^{-\alpha}}{2} (x_g - x_e)^2 + \frac{1}{2} \alpha \quad (27)$$

提醒:

- σ 是网络预测的标准方差;
- 狄拉克函数具体挑选性: $\int f(x) \delta(x - t_0) dx = f(t_0)$ 【见公式 (22)】;
- 可以理解为所有 $iou(m, b_i) \geq N_t$ 的候选框预测坐标的加权平均。

2.3.5 Fast NMS

每个类别独立执行的, 基本流程为:

- 生成 IoU 矩阵: 为当前类别的检测框计算 IoU 矩阵
- 初始化: 将 IoU 矩阵的下三角和对角线设置为 0
- 计算最大 IoU: 对 IoU 矩阵的每一列计算最大值, 得到矩阵 K
- 阈值过滤: 根据阈值 t 决定哪些检测框被保留, 哪些被抑制
- 输出结果: 输出该类别的最终检测框列表

2.3.6 Weighted-Boxes-Fusion, WBF

提醒:

- 与单一模型相比, 结合多个模型的预测会更好
- NMS 和 soft-NMS 只是简单地删除部分预测框
- 比 NMS 慢 3 倍

具体的流程, 可总结为: 维护两个容器 Clusters 和 Fusions【一个 Fusion 对应一簇 Cluster】, 详细步骤如下:

Algorithm 2: WBF

输入: 候选框集合 $\mathcal{B}$, 按其对应的置信度 $\mathcal{S}$ 降序排列; 检测结果容器 $\mathcal{L}$ 和融合容器 $\mathcal{F}$, 阈值 N_t
最终检测结果 $\mathcal{D} \leftarrow \{\}$

while $\mathcal{B} \neq \text{empty}$ **do**

 步骤一, 按顺序从 $\mathcal{B}$ 中取出一个框;

 步骤二, 从 $\mathcal{F}$ 中找到与之匹配的框, 要求属于同一类别且 $\text{mIoU} > N_t$ 。如果没有找到, 则将其加到 $\mathcal{L}$ 和 $\mathcal{F}$ 的尾部; 反之, 则将其加到 $\mathcal{L}$, 加入位置为其在 $\mathcal{F}$ 中匹配到的位置;

 步骤三, 重新计算 $\mathcal{F}$ 中的 box 坐标和置信度分数, 融合 $\mathcal{L}$ 中的所有 box, 融合公式为:

$$\mathcal{S} = \frac{\sum_{i=1}^T \mathcal{S}_i}{T} \quad (28)$$

$$\mathcal{X}_{1,2} = \frac{\sum_{i=1}^T \mathcal{S}_i * \mathcal{X}_{1,2_i}}{\sum_{i=1}^T \mathcal{S}_i} \quad (29)$$

$$\mathcal{Y}_{1,2} = \frac{\sum_{i=1}^T \mathcal{S}_i * \mathcal{Y}_{1,2_i}}{\sum_{i=1}^T \mathcal{S}_i} \quad (30)$$

 步骤四, 将 $\mathcal{B}$ 中所有的框都处理完后, 重新缩放 $\mathcal{F}$ 中的置信度分数:

$$\mathcal{C} = \mathcal{C} * \frac{\min(T, N)}{N} \quad (31)$$

 如果簇中的框数少, 则意味着只有少数模型可以预测它, 因此需要降低这种情况的置信度分数

end

输出: 最终检测结果

3 RCNN 系列

3.1 RCNN

3.2 Fast RCNN

3.3 Faster RCNN

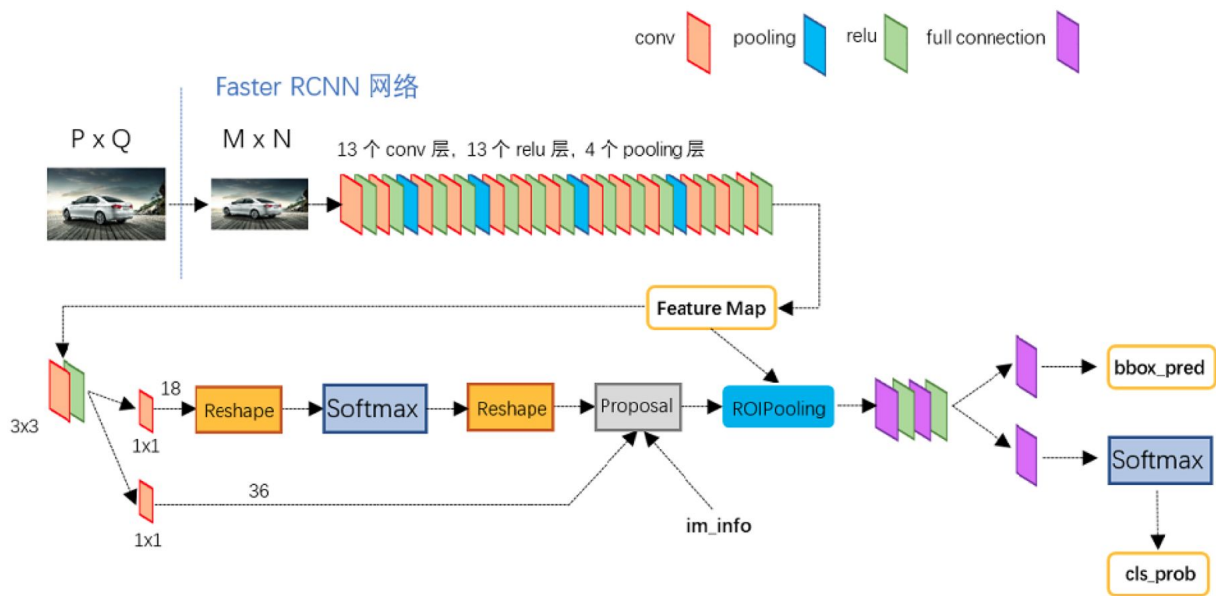


图 3: Faster RCNN 网络架构

Two-stages 中最为经典的物体检测算法。具体地，大致可以分为四个部分：

3.3.1 Backbone

以 VGG16、Resnet101 等为主。输入 RGB 图片数据，压缩 16 倍【4 次下采样】，输出 Feature maps；

3.3.2 Head

faster rcnn 包括两个 head：rpn head 和 rcnn head。其结构如下：

```
rpn_head=dict(  
    type=RPNHead,  
    in_channels=256,  
    feat_channels=256,  
    anchor_generator=dict(  
        type=AnchorGenerator,  
        scales=[8],  
        ratios=[0.5, 1.0, 2.0],  
        strides=[4, 8, 16, 32, 64]),  
    bbox_coder=dict(  
        type=DeltaXYWHBBoxCoder,
```

```

        target_means=[.0, .0, .0, .0],
        target_stds=[1.0, 1.0, 1.0, 1.0]),
    loss_cls=dict(
        type=CrossEntropyLoss, use_sigmoid=True, loss_weight=1.0),
    loss_bbox=dict(type=L1Loss, loss_weight=1.0)),

```

```

bbox_head=dict(
    type=Shared2FCBBoxHead,
    in_channels=256,
    fc_out_channels=1024,
    roi_feat_size=7,
    num_classes=80,
    bbox_coder=dict(
        type=DeltaXYWHBBoxCoder,
        target_means=[0., 0., 0., 0.],
        target_stds=[0.1, 0.1, 0.2, 0.2]),
    reg_class_agnostic=False,
    loss_cls=dict(
        type=CrossEntropyLoss, use_sigmoid=False, loss_weight=1.0),
    loss_bbox=dict(type=L1Loss, loss_weight=1.0))

```

rpn head 的输出是包括分类和回归，分类是二分类，只区分前景和背景；回归是仅仅对于前景样本（正样本）进行基于 anchor 的变换回归。rpn head 的目的是提取 roi，然后输入到 rcnn head 部分进行 refine。rcnn head 的输出是包括分类和回归，分类输出是类别数 +1（1 是考虑背景），回归是仅仅对于前景样本不考虑分类类别进行基于 roi 的变换回归，rcnn head 的目的是对 rpn 提取的 roi 特征进行 refine，输出精准 bbox。

提醒： 正负样本定义

rpn 和 rcnn 的正负样本定义都是基于 MaxIoUAssigner，只不过定义阈值不一样而已。

```

rpn=dict(
    assigner=dict(
        type=MaxIoUAssigner,
        pos_iou_thr=0.7,
        neg_iou_thr=0.3,
        min_pos_iou=0.3,
        match_low_quality=True,
        ignore_iof_thr=-1)
    ),
    rcnn=dict(
        assigner=dict(
            type=MaxIoUAssigner,
            pos_iou_thr=0.5,
            neg_iou_thr=0.5,
            min_pos_iou=0.5,
            match_low_quality=False,
            ignore_iof_thr=-1)
        ),

```

提醒：MaxIoUAssigner 进行详细分析

共 4 个步骤：

- 初始化时候假设每个 anchor 的 mask 都是-1，表示都是忽略 anchor
- 将每个 anchor 和所有 gt 的 iou 的最大 Iou 小于 neg_iou_thr 的 anchor 的 mask 设置为 0，表示是负样本（背景样本）
- 对于每个 anchor，计算其和所有 gt 的 iou，选取最大的 iou 对应的 gt 位置，如果其最大 iou 大于等于 pos_iou_thr，则设置该 anchor 的 mask 设置为 1，表示该 anchor 负责预测该 gt bbox，是高质量 anchor
- 3 的设置可能会出现某些 gt 没有分配到对应的 anchor（由于 iou 低于 pos_iou_thr），故下一步对于每个 gt 还需要找出和最大 iou 的 anchor 位置，如果其 iou 大于 min_pos_iou，将该 anchor 的 mask 设置为 1，表示该 anchor 负责预测对应的 gt。通过本步骤，可以**最大程度保证每个 gt 都有 anchor 负责预测，如果还是小于 min_pos_iou，那就没办法了，只能当做忽略样本了**。从这一步可以看出，3 和 4 有部分 anchor 重复分配了，即当某个 gt 和 anchor 的最大 iou 大于等于 pos_iou_thr，那肯定大于 min_pos_iou，此时 3 和 4 步骤分配的同一个 anchor

在第 4 步中，每个 gt 最多只会和某一个 anchor 匹配，但是实际操作时候为了多增加一些正样本，通过参数 gt_max_assign_all 可以实现某个 gt 和多个 anchor 匹配场景。**通常第 4 步引入的都是低质量 anchor，网络训练有时候还会带来噪声，可能还会起反作用。**

由于 rcnn head 预测值是 rpn head 的 refine，故 rcnn head 面对的 anchor（其实就是 rpn 输出的 roi）和 gt 的 iou 会高于 rpn head 部分，anchor 质量更高，故 min_pos_iou 阈值设置的比较高，由于 pos_iou_thr 和 neg_iou_thr 设置都是 0.5，那么忽略区域那就是没有了，因为 rcnn head 面对的都是高质量样本，不应该还存在忽略区域。

3.3.3 Region Proposal Networks

Region Proposal Networks (RPN) 用于生成 region proposals **【可以理解为在原图尺度上，设置密密麻麻的候选 anchors，然后用 CNN 判断哪些 anchors 里面有目标，哪些没有】**【一张 $M \times N$ 的图，一共有 $\text{ceil}(M/16) \times \text{ceil}(N/16) \times \text{num}_{anchors}$ 个 anchors】**【全部 anchors 拿去训练太多了，随机选取 128 个正例和 128 个负例】**。

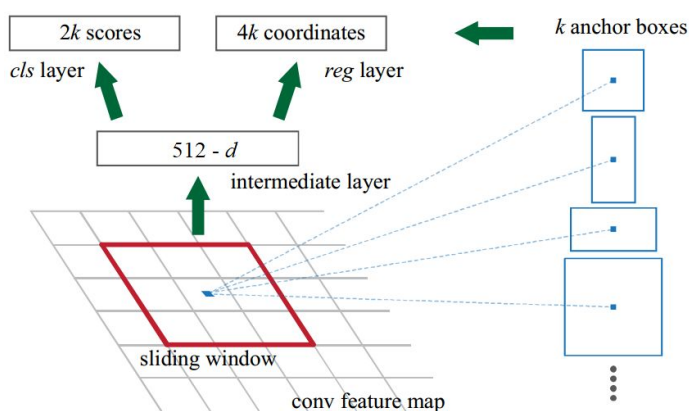


图 4: Anchors 示意图

feature map 经过 512 个 3×3 的卷积核之后得到 512 个通道的特征图。

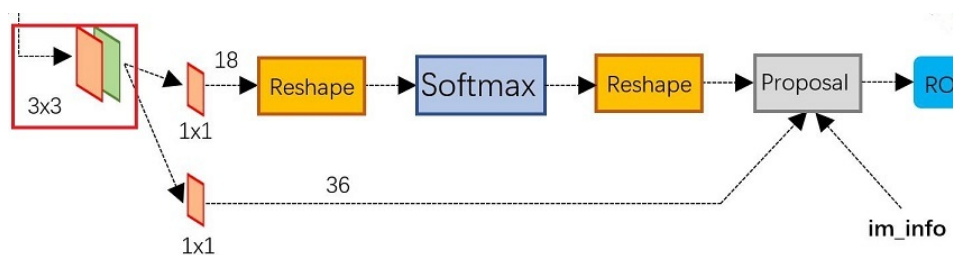


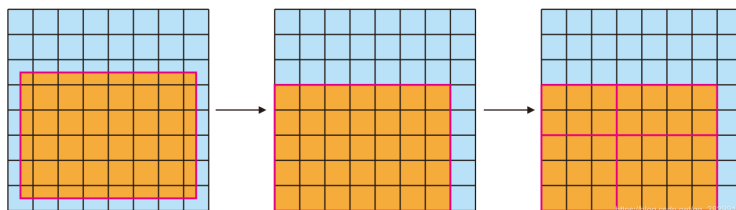
图 5: RPN 结构示意图

- 图5上面一层 (图4左边) **512-dimensions** 【不是 512 减 d】特征图通过 softmax 【softmax 常和 sigmoid 对比, 前者计算梯度简单, 但要求类别相互独立】转化为 $cls = 2k$, 并判断 anchors 属于正例还是负例。具体的判断标准如下:
 - 与 Ground truth 的 $IoU > 0.7$, 记为正例;
 - 与 Ground truth 的 $IoU < 0.3$, 记为负例
- 图5下面一层 (图4右边) 利用 bounding box regression 修正 anchors 获得 proposals, 并剔除太小和超出边界的 proposals 【实现目标定位】;

bounding box regression 原理可参考4.3.4节。

3.3.4 RoI Pooling

收集输入的 feature maps 和 proposals, 综合这些信息后提取 proposal feature maps, 送入后续全连接层判定目标类别

图 6: RoI Pooling 输出为 2×2 时的流程图**提醒:**

- 为什么需要 RoI Pooling ?

后面接全连接层要求固定输入的大小。因此当大小不定时, 要么从图像中 crop 一部分【破坏了图像的完整性】, 要么将图像 warp 为需要的大小【破坏了图像的形状】, 可参考 Spatial Pyramid Pooling

- 不匹配问题! 见图6

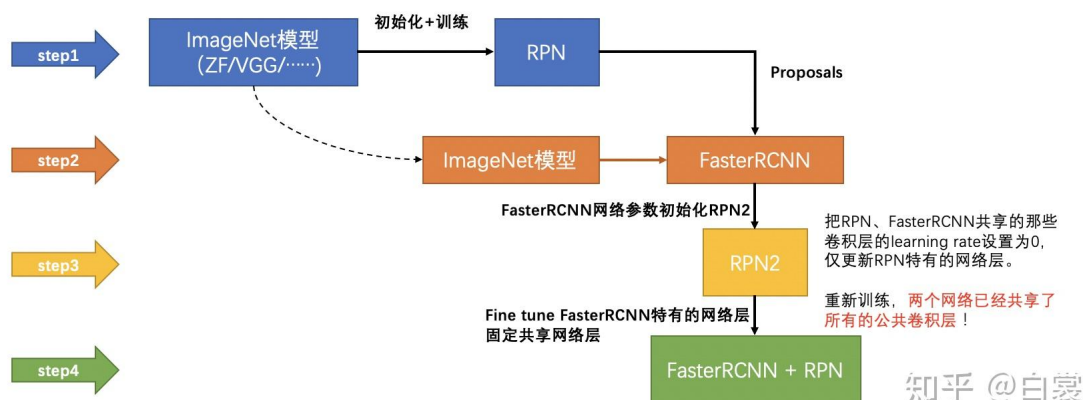
- Region proposal 的 xywh 通常是小数, 但为了方便操作会将其整数化;
- 将整数化后的边界区域平均分割为 $k \times k$ 个单元, 还需要对每个单元的边界整数化。

此时, 候选框和最开始的位置存在一定的偏差。【在3.4节中提出了相应的解决办法】

3.3.5 Classification

计算 proposal 的类别, 并再次 bounding box regression 获得检测框的精确位置

以上整个流程如下:

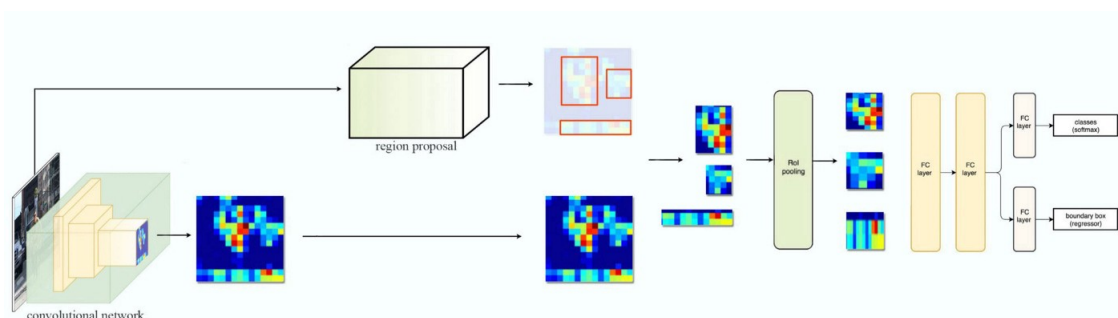


1. 在已经训练好的 model (VGG) 上, 训练 RPN 网络;
2. 在步骤 1 的基础上, 收集 proposals;
3. 训练 Faster RCNN;

4. 第二次训练 RPN;
5. 在步骤 4 的基础上, 收集 proposals;
6. 第二次训练 Faster RCNN

可以看到训练过程类似于一种“迭代”的过程, 不过只循环了 2 次。至于只循环了 2 次的原因是作者提到的: ”A similar alternating training can be run for more iterations, but we have observed negligible improvements”, 即循环更多次没有提升了

3.3.6 推理阶段



3.4 Mask RCNN

4 YOLO 系列

4.1 YOLOV1

4.2 YOLOV2

YOLOv2 在 YOLOv1 的基础上进行了改进, 引入了一些新的技术, 如 Anchor Boxes 和 Darknet-19 等。Anchor Boxes 可以更好地处理不同尺寸和比例的目标, 而 Darknet-19 是一种更深的神经网络结构, 提升了检测的准确性。此外, YOLOv2 还引入了多尺度训练和预测, 进一步提高了检测性能。

4.3 YOLOV3

YOLOv3 是对 YOLOv2 的进一步改进, 主要包括使用更大的网络结构 Darknet-53、使用多尺度预测和采用更多的 Anchor Boxes。YOLOv3 在检测精度和速度方面都有显著提升, 并且能够更好地处理小目标。

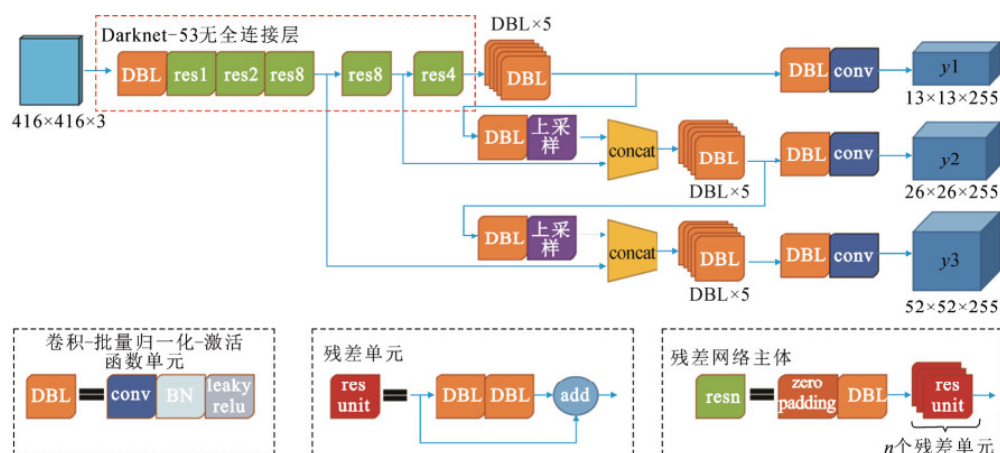


图 7: YOLOV3 结构图

输入: $(bs, 416, 416, 3)$

具体地, 将原图按纵横比例缩放至 416×416 , 取 $\min(w/img_w, h/img_h)$ 保证图片不会出现扭曲, 多余的部分用 $(128, 128, 128)$ 填充。相应地, 标签也需要进行相应调整以及归一化处理【避免回归框时, 大边框过度主导梯度方向】

```
# annotation_line: ROOT/000005.jpg 263,211,324,339,1
def get_random_data(self, annotation_line, input_shape, jitter=.3, hue=.1, sat=0.7, val=0.4, random=True):
    line = annotation_line.split()
    image = Image.open(line[0])
    image = cvtColor(image) # 读取图像并转换成RGB图像
    iw, ih = image.size # 获取图像的宽高
    h, w = input_shape # 获取目标的宽高
    # 获取目标框信息
    box = np.array([np.array(list(map(int, box.split(',')))) for box in line[1:]])
    if not random:
        scale = min(w/iw, h/ih)
        nw = int(iw*scale)
        nh = int(ih*scale)
        dx = (w-nw)//2
        dy = (h-nh)//2
        image = image.resize((nw,nh), Image.BICUBIC)
        new_image = Image.new('RGB', (w,h), (128,128,128)) # 多余的部分加上灰条
        new_image.paste(image, (dx, dy))
        image_data = np.array(new_image, np.float32)

    # 调整真实框
    if len(box) > 0:
        np.random.shuffle(box)
        box[:, [0,2]] = box[:, [0,2]]*nw/iw + dx
        box[:, [1,3]] = box[:, [1,3]]*nh/ih + dy
        box[:, 0:2][box[:, 0:2]<0] = 0
        box[:, 2][box[:, 2]>w] = w
        box[:, 3][box[:, 3]>h] = h
        box_w = box[:, 2] - box[:, 0]
        box_h = box[:, 3] - box[:, 1]
```

```
box = box[np.logical_and(box_w>1, box_h>1)] # 丢弃无效框
return image_data, box
```

4.3.1 Darknet53

Darknet53 【要求输入图片的大小是 32 的倍数】，包含 53 个卷积层，没有池化层 【4 次下采样通过卷积操作实现。卷积相较于池化，前者更注重保留信息，且增加了非线性变换的次数，后者更注重筛选信息】

	类型	卷积核数量	卷积核尺寸	输出
	卷积层	32	3×3	256×256
	卷积层	64	3×3/2	128×128
1×	卷积层	32	1×1	128×128
	卷积层	64	3×3	
	残差层			
	卷积层	128	3×3/2	
2×	卷积层	64	1×1	64×64
	卷积层	128	3×3	
	残差层			
	卷积层	256	3×3/2	
8×	卷积层	128	1×1	32×32
	卷积层	256	3×3	
	残差层			
	卷积层	512	3×3/2	
8×	卷积层	256	1×1	16×16
	卷积层	512	3×3	
	残差层			
	卷积层	1024	3×3/2	
4×	卷积层	512	1×1	8×8
	卷积层	1024	3×3	
	残差层			
	卷积层			

图 8: Darknet-53 结构图。残差结构缓解梯度消失的问题，同时复用特征，强化学习网络

Backbone 输出： $bs \times 52 \times 52 \times 256$ 、 $bs \times 26 \times 26 \times 512$ 、 $bs \times 13 \times 13 \times 1024$ 三类特征图 【小尺度特征图用于检测大尺寸物体；大尺寸特征图用于检测小尺度特征图】

4.3.2 Neck

由五个卷积层组成，负责生成特征金字塔。具体地：将 $bs \times 13 \times 13 \times 1024$ 的特征图上采样变为 $bs \times 26 \times 26 \times 256$ ，再与 $bs \times 26 \times 26 \times 512$ 堆叠，得到维度为 $bs \times 26 \times 26 \times 768$ 的特征图，经由五个卷积层后，变为 $bs \times 26 \times 26 \times 256$ 。

4.3.3 Head

由两个卷积层 【卷积核分别为 3×3 、 1×1 ，后一个卷积操作改变通道数的同时实现预测】组成，输出结果分别为 $bs \times 13 \times 13 \times (3 * (4 + 1 + cls_num))$ 、 $bs \times 26 \times 26 \times (3 * (4 + 1 + cls_num))$ 、 $bs \times 52 \times 52 \times (3 * (4 + 1 + cls_num))$

4.3.4 推理阶段

$$bs \times N \times N \times [3 \times (4 + 1 + cls_num)]$$

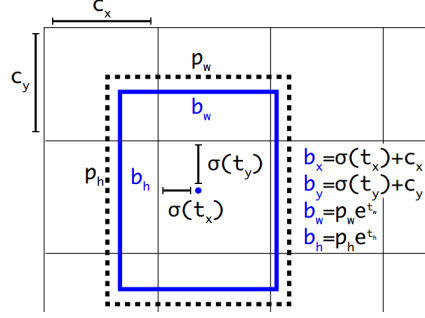


图 9: 检测框解码

预测结果 (t_x, t_y, t_w, t_h) 解码【中心点平移到蓝色点的过程】：

网络输出获得边界框预测：

$$b_x = (\sigma(t_x) + c_x) / W \quad (32)$$

$$b_y = (\sigma(t_y) + c_y) / H \quad (33)$$

$$b_w = p_w e^{t_w} / W \quad (34)$$

$$b_h = p_h e^{t_h} / H \quad (35)$$

其中, (c_x, c_y) 为网格从顶左部的坐标; p_w, p_h 分别是锚框的维度; W, H 分别是特征图的维度。

```
# 真实框标签化
# b: batch-size; k: 第k+1个框; (i, j): 网格点坐标; t: 第t+1个真实框
y_true[b, k, j, i, 0] = batch_target[t, 0] - i.float()
y_true[b, k, j, i, 1] = batch_target[t, 1] - j.float() # 不需要除以anchors[best_n]的宽高
y_true[b, k, j, i, 2] = math.log(batch_target[t, 2] / anchors[best_n][0])
y_true[b, k, j, i, 3] = math.log(batch_target[t, 3] / anchors[best_n][1]) # 消除框的大小对误差的影响
y_true[b, k, j, i, 4] = 1 # 置信度标签
y_true[b, k, j, i, c + 5] = 1 # 类别标签
```

在 faster-RCNN 中, t_x, t_y, t_w, t_h 依次可以表示为:

$$t_x = (G_x - P_x) / P_w \quad (36)$$

$$t_y = (G_y - P_y) / P_h \quad (37)$$

$$t_w = \log(G_x / P_x) \quad (38)$$

$$t_h = \log(G_y / P_y) \quad (39)$$

其中, (P_x, P_y) 和 (G_x, G_y) 分别为预设的 anchor box 和 ground truth 在特征图上的中心点坐标; (P_w, P_h) 和 (G_w, G_h) 分别为预设的 anchor box 和 ground truth 在特征图上的宽和高【归一化变绝对尺度为相对尺度】。

提醒：

- 没有直接回归 bounding box 的长宽而是尺度缩放到对数空间，去掉不等式约束【相对形变必须大于 0】
- 当输入的 Proposal 与 ground truth 相差较小时，可以认为边框回归是一种线性变换

$$t_w = \log(G_w/P_w) = \log\left(1 + \frac{G_w - P_w}{P_w}\right) = \frac{G_w - P_w}{P_w} (P_w - G_w \rightarrow 0)$$

- sigmoid 压缩 t_x, t_y 到 (0, 1)，可以有效地确保目标中心位于预测的网格单元中

提醒： 关于多尺度特征图的标签分配：

- YOLOV3 不像 YOLOV1 【每个 grid 负责中心落在该 grid 中的 ground truth；YOLOV3 三类特征图可能会存在中心重合】
- 两种分配思路：
 - i>. 先根据 ground truth 的大小和所有 anchor 的 IOU 分配合适的 anchor 作为正例

```
# gt_box:(num_true_box, 4)
# anchor_shapes: (9, 4)
# best_ns: [每个真实框最大的重合度, 每一个真实框最重合的先验框的序号]
best_ns = torch.argmax(self.calculate_iou(gt_box, anchor_shapes), dim=-1)
# 为 每一个真实框最重合的先验框 适配对应尺度的特征图, 1: 0/1/2
for t, best_n in enumerate(best_ns):
    if best_n not in self.anchors_mask[1]: continue
```

- ii>. 直接在 3 种尺度上，选 ground truth 对应 IOU 最大的 anchor

提醒： 预测框【总共 $\sum_{i=0}^3 3 \times N_i \times N_i$ 】的三种情况

- 正例：任取一个 ground truth，与所有的预测框计算 IOU，IOU 最大的预测框即为正例【一个预测框只能分配一个 ground truth；正例计算检测框 loss、置信度 loss、类别 loss】；
- 忽略样例：正例除外，与任意一个 ground truth 的 IOU 大于阈值，则为忽略样例【忽略样本样例不产生任何 loss】；

```
# obj_mask: obj_mask = y_true[..., 4] == 1, 其他为False
# noobj_mask: 正例对应的noobj_mask[b, k, j, i] = 0; noobj_mask[b][anch_ious_max > self.
    ignore_threshold] = 0, 其他为1, 即为负例
# 只有正例和负例计算置信度损失
loss_conf = torch.mean(self.BCELoss(conf, obj_mask.type_as(conf))[noobj_mask.bool() | obj_mask
    ])
```

- 负例：正例除外，与全部 ground truth 的 IOU 都小于阈值，则为负例【正例样本与 ground truth 的 IOU 可能小于 0.5，不冲突；只计算置信度 loss】

损失函数：由四个部分组成：

- 边界框损失：

在 YOLOV1 中，计算长宽损失**多一个根号**，【对于相同的误差值，大物体误差对检测的影响应小于小物体误差对检测的影响。例如，5 个像素点的偏差，对于 416×416 的预测框几乎没有影响，但是对于 10×10 的预测框影响就很大】【但是加了根号后对小框的损失来说有些过大。例如 $(\sqrt{20} - \sqrt{10})^2 = 3.43$ 、 $(\sqrt{110} - \sqrt{100})^2 = 0.48$ 】；

在 YOLOV3 中，用以下方法控制不同大小的目标对应的权重：

```
box_loss_scale[b, k, j, i] = batch_target[t, 2] * batch_target[t, 3] / in_w / in_h
box_loss_scale = 2 - box_loss_scale
```

以上是计算的回归损失【与性能评估指标存在不一致性】，现阶段多采用 IOU 及其变体进行计算。这样做也更加有利于目标检测【大多数对象是相对于图像本身而言的小目标体】，具体详见附录。

- 置信度损失：可以理解为粗划分【二分类】，另外由于检测框输出结果存在明显的类别失衡问题【大部分为负例】，在4.4节改 BCE 为 Focal loss2.1.1。
- 类别预测损失

4.3.5 后处理

```
class Detect(nn.Module):
    # YOLOv3 Detect head for detection models
    stride = None # strides computed during build
    dynamic = False # force grid reconstruction
    export = False # export mode

    def __init__(self, nc=80, anchors=(), ch=(), inplace=True): # detection layer
        super().__init__()
        self.nc = nc # number of classes
        self.no = nc + 5 # number of outputs per anchor
        self.nl = len(anchors) # number of detection layers
        self.na = len(anchors[0]) // 2 # number of anchors
        self.grid = [torch.empty(0) for _ in range(self.nl)] # init grid
        self.anchor_grid = [torch.empty(0) for _ in range(self.nl)] # init anchor grid
        self.register_buffer("anchors", torch.tensor(anchors).float().view(self.nl, -1, 2)) # shape(nl,na,2)
        self.m = nn.ModuleList(nn.Conv2d(x, self.no * self.na, 1) for x in ch) # output conv
        self.inplace = inplace # use inplace ops (e.g. slice assignment)

    def forward(self, x):
        z = [] # inference output
        for i in range(self.nl):
            x[i] = self.m[i](x[i]) # conv
            bs, _, ny, nx = x[i].shape # x(bs,255,20,20) to x(bs,3,20,20,85)
            x[i] = x[i].view(bs, self.na, self.no, ny, nx).permute(0, 1, 3, 4, 2).contiguous()

        if not self.training: # inference
```

```

        if self.dynamic or self.grid[i].shape[2:4] != x[i].shape[2:4]:
            self.grid[i], self.anchor_grid[i] = self._make_grid(nx, ny, i)

        xy, wh, conf = x[i].sigmoid().split((2, 2, self.nc + 1), 4)
        xy = (xy * 2 + self.grid[i]) * self.stride[i] # xy
        wh = (wh * 2) ** 2 * self.anchor_grid[i] # wh
        y = torch.cat((xy, wh, conf), 4)
        z.append(y.view(bs, self.na * nx * ny, self.no))

    return x if self.training else (torch.cat(z, 1),) if self.export else (torch.cat(z, 1), x)

def _make_grid(self, nx=20, ny=20, i=0, torch_1_10=check_version(torch.__version__, "1.10.0")):
    d = self.anchors[i].device
    t = self.anchors[i].dtype
    shape = 1, self.na, ny, nx, 2 # grid shape
    y, x = torch.arange(ny, device=d, dtype=t), torch.arange(nx, device=d, dtype=t)
    yv, xv = torch.meshgrid(y, x, indexing="ij") if torch_1_10 else torch.meshgrid(y, x) # torch>=0.7
    compatibility
    grid = torch.stack((xv, yv), 2).expand(shape) - 0.5 # add grid offset, i.e. y = 2.0 * x - 0.5
    anchor_grid = (self.anchors[i] * self.stride[i]).view((1, self.na, 1, 1, 2)).expand(shape)
    return grid, anchor_grid

```

提醒：边框回归里面 wh 为什么要乘以 2 再平方

主要是因为原始的框方程式存在缺陷，宽高虽然恒大于 0，但不受限制，容易梯度不稳定，增加训练难度，因此使用 $(2\sigma(t_w))^2$ ，这样既能保证宽高恒大于 0，也能对宽高加以限制，最大值为 anchor 框宽高的 4 倍

4.4 YOLOV4

YOLOv4 是 YOLO 系列中的最新版本，它在 YOLOv3 的基础上引入了一系列新的技术和优化策略。YOLOv4 采用了更大的网络结构 CSPDarknet53，引入了 SAM (Spatial Attention Module) 和 PANet (Path Aggregation Network) 等模块，提升了模型的感知能力和特征融合能力。此外，YOLOv4 还采用了一些训练和推理技巧，如 Mosaic 数据增强、CIoU 损失函数和 CIOU-NMS 等，进一步提升了检测的准确性和速度。

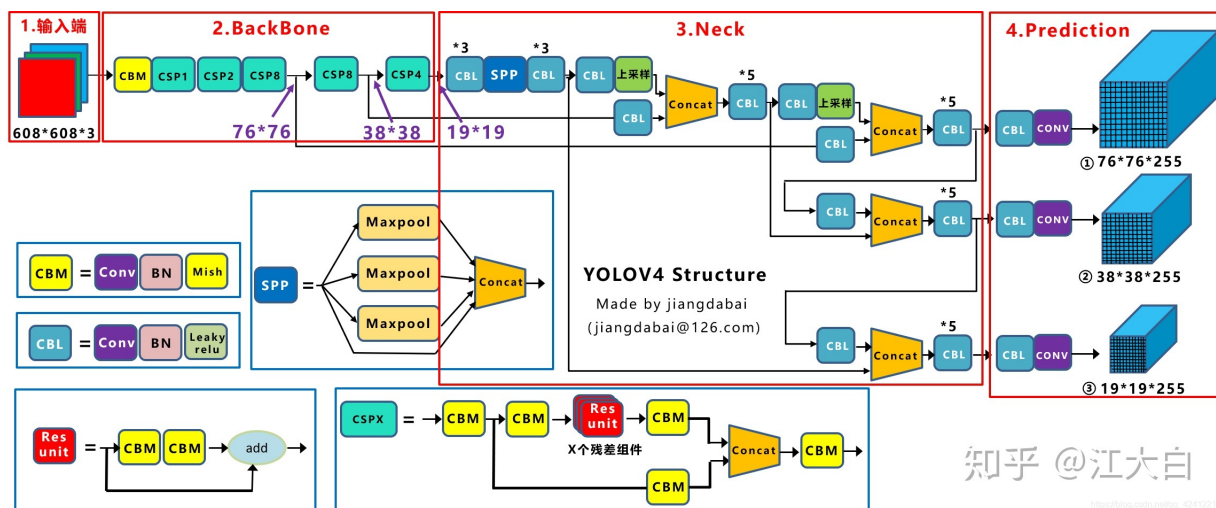


图 10: YOLOV4 结构图

4.4.1 CSPDarknet53

CSPDarknet53: Cross Stage Parital Network (CSP): 【网上写得太花哨】, 可以参考 GoogLeNet, 优点可以总结为两点:

提醒:

- 特征复用, 强化学习;
- 降低内存成本

4.4.2 SPP 模块

采用 SPP 模块的方式, 比单纯的使用 $k \times k$ 最大池化的方式, 更有效的增加主干特征的接收范围, 显著的分了最重要的上下文特征

4.4.3 Feature pyramid network

Feature pyramid network (FPN) 是 CVPR2017 年的一篇文章, 它在目标检测中融入了特征金字塔, 提高了目标检测的准确率, 尤其体现在小物体的检测上。特征金字塔的深层特征包含有丰富的语义信息 (利于分类), 但分辨率低, 很难准确地表达物体的位置信息; 浅层特征虽然语义信息较少, 但分辨率高, 包含准确的物体位置信息 (利于检测和分割)。FPN 将浅层特征和深层特征融合起来, 得到一个识别和定位都准确的目标检测结构。FPN 的结构主要包括三个部分: Bottom-up, Top-down 和 Lateral connection, 如下图所示。

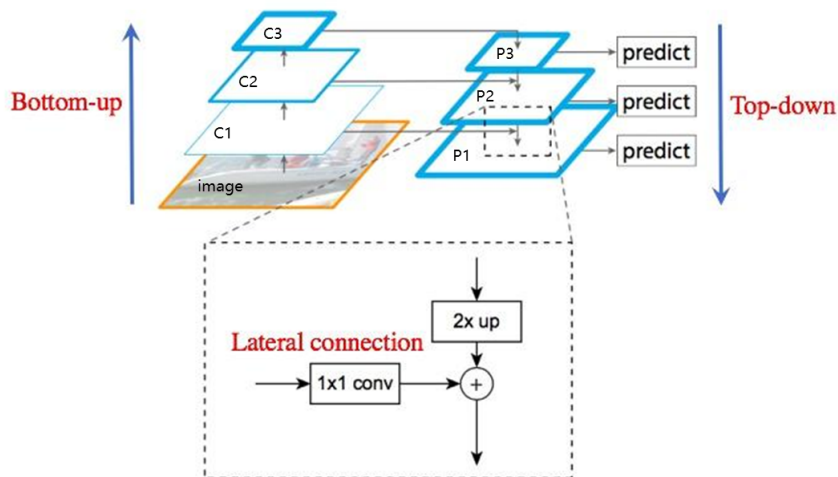


图 11: FPN 结构图

4.4.4 Path Aggregation Network

Path Aggregation Network (PANet) 是一个用于实例分割的路径聚合网络，它充分的融合了特征。共有 5 个部分，如下图所示，分别是 FPN(a)、Bottom-Up Path Augmentation(b)、Adaptive Feature Pooling(c)、Box branch(d)、Fully-Connected Fusion(e)。

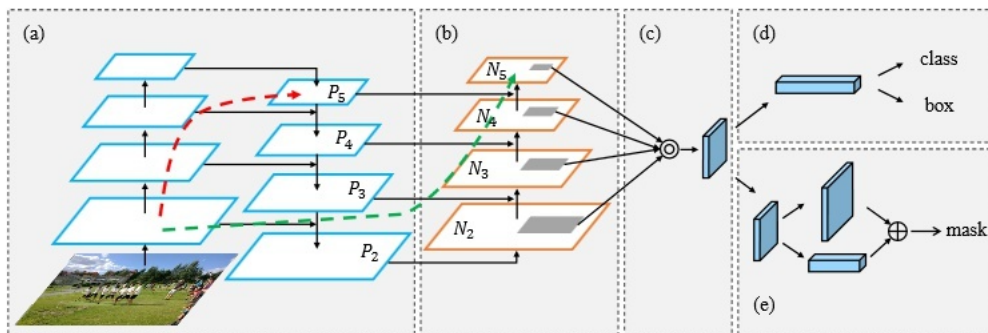


Figure 1. Illustration of our framework. (a) FPN backbone. (b) Bottom-up path augmentation. (c) Adaptive feature pooling. (d) Box branch. (e) Fully-connected fusion. Note that we omit channel dimension of feature maps in (a) and (b) for brevity.

图 12: PANet 结构图

- FPN 上文已经介绍，这里不再赘述；
- Bottom-Up Path Augmentation: 考虑到网络的浅层特征对于实例分割非常重要，FPN 虽然已经融合了一次浅层特征，但是仍不能达到很好的分割效果，因此引入 Bottom-Up Path Augmentation，以保留更多的浅层特征。它是将 N_2 (N_2 和 P_2 是同一特征图，因此 N_2 已经包含大量低层特征) 进行下采样，下采样后的特征图与 P_3 进行融合，得到 N_3 。因此 N_3 包含的低层特征要多与 P_3 。 N_4 , N_5 同 N_3 ；
- Adaptive Feature Pooling: 这一结构做的仍然是特征融合；


```
weights /= weights.sum() # 赋予每个类别的权重为该类别的目标框数
return torch.from_numpy(weights).float()
```

2. 结合类别权重，计算训练样本中各个输入图片的权重；

```
def labels_to_image_weights(labels, nc, class_weights=np.ones(nc)):
    weights = image_weights, k=1) # weighted image sample
    class_counts = np.array([np.bincount(x[:, 0].astype(int), minlength=nc) for x in labels])
    return (class_weights.reshape(1, nc) * class_counts).sum(1) # 图片中每个标签的权重之和即为图片的权重
```

3. 依据图片权重采样，训练

```
.....
for epoch in range(start_epoch, epochs):
    .....
    # 持续更新
    if opt.image_weights:
        # 初始化maps = np.zeros(nc)
        # 标签数量越多，且标签识别精度越高，赋予的标签的权重越小
        cw = model.class_weights.cpu().numpy() * (1 - maps) ** 2 / nc
        iw = labels_to_image_weights(dataset.labels, nc=nc, class_weights=cw) # 图像权重
        dataset.indices = random.choices(range(dataset.n), weights=iw, k=dataset.n)
    .....
```

4.5.2 推理阶段

```
def __yolov5_decode(self,
                    feats: List[ndarray],
                    conf_thres: float,
                    num_labels: int = 80,
                    **kwargs):
    anchors: Union[List, Tuple] = kwargs.get(
        'anchors',
        [[(10, 13), (16, 30),
          (33, 23)], [(30, 61), (62, 45),
          (59, 119)], [(116, 90), (156, 198), (373, 326)]]
    )
    for i, feat in enumerate(feats):
        stride = 8 << i
        feat_h, feat_w, _ = feat.shape
        anchor = anchors[i]
        feat = sigmoid(feat)
        feat = feat.reshape((feat_h, feat_w, len(anchor), -1))
        box_feat, conf_feat, score_feat = np.split(feat, [4, 5], -1)

        hIdx, wIdx, aIdx, _ = np.where(conf_feat > conf_thres)

        num_proposal = hIdx.size
        if not num_proposal:
```

```

        continue

    score_feat = score_feat[hIdx, wIdx, aIdx] * conf_feat[hIdx, wIdx, aIdx]
    boxes = box_feat[hIdx, wIdx, aIdx]
    labels = score_feat.argmax(-1)
    scores = score_feat.max(-1)

    indices = np.where(scores > conf_thres)[0]
    if len(indices) == 0:
        continue

    for idx in indices:
        a_w, a_h = anchor[aIdx[idx]]
        x, y, w, h = boxes[idx]
        x = (x * 2.0 - 0.5 + wIdx[idx]) * stride
        y = (y * 2.0 - 0.5 + hIdx[idx]) * stride
        w = (w * 2.0)**2 * a_w
        h = (h * 2.0)**2 * a_h

        x0 = x - w / 2
        y0 = y - h / 2

        self.scores_pro.append(float(scores[idx]))
        self.boxes_pro.append(
            np.array([x0, y0, w, h], dtype=np.float32))
        self.labels_pro.append(int(labels[idx]))

```

提醒： 和 yolov3 相比，yolov5 的边框回归方式有什么不同之处

yolov5 先对 (t_x, t_y) sigmoid 处理后，再对其两倍计算后减去 0.5。这与 yolov5 正样本的定义有关。yolov5 和 yolov3/v4 有两点不同：

- 匹配规则：从 IOU 匹配变成 shape 匹配，先计算 gt 与 9 个 anchor 的长宽比，如果长宽比小于设定阈值，说明该 gt 和对应的 anchor 匹配，一个 gt 可能与几个 anchor 均能匹配上，因为同之前 yolo 一样，yolov5 有三层网络，9 个 anchor，从小到大，每 3 个 anchor 对应一层网络，所以一个 gt 可能在不同的网络层上做预测训练，大大增加了正样本的数量，当然也会出现 gt 与所有 anchor 都匹配不上的情况，这样 gt 就会被当成背景，不参与训练，说明 anchor 框尺寸设计的不好
- 筛选框：gt 框与 anchor 框匹配后，得到 anchor 框对应的网络层的 grid，看 gt 中心点落在哪个 grid 上，不仅取该 grid 中和 gt 匹配的 anchor 作为正样本，还取相邻的两个 grid 中的 anchor 为正样本，增大正样本数量

4.6 YOLOX

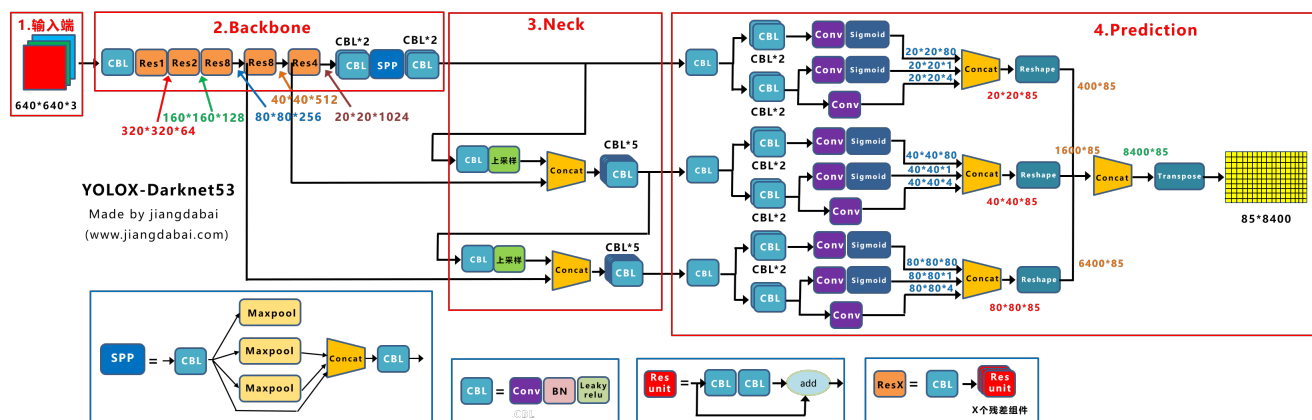


图 14: YOLOX 网络结构图

4.6.1 Strong augmentation

在网络的输入端, YOLOX 主要采用了 Mosaic、Mixup 两种数据增强方法。而采用了这两种数据增强, 直接将 YOLOv3 baseline, 提升了 2.4 个百分点。

- Mosaic 增强的方式, 是 U 版 YOLOv3 引入的一种非常有效的增强策略。而且在 YOLOv4、YOLOv5 算法中, 也得到了广泛的应用。通过随机缩放、随机裁剪、随机排布的方式进行拼接, 对于小目标的检测效果提升, 还是很不错的。
- MixUp 是在 Mosaic 基础上, 增加的一种额外的增强策略。主要来源于 2017 年, 顶会 ICLR 的一篇论文《mixup: Beyond Empirical Risk Minimization》。当时主要应用在图像分类任务中, 可以在几乎无额外计算开销的情况下, 稳定提升 1 个百分点的分类精度。

提醒: 需要注意两点:

- 在训练的最后 15 个 epoch, 这两个数据增强会被关闭掉。而在此之前, Mosaic 和 Mixup 数据增强, 都是打开的;
- 由于采取了更强的数据增强方式, 作者发现, ImageNet 预训练将毫无意义, 因此, 所有的模型, 均是从头开始训练的

提醒： 无节制的数据增强是否真的更好呢？

- Mosaic 和 Mixup 混合策略：对于轻量级网络，比如 YOLOX-nano，在 Mosaic 基础上，增加 Mixup，AP 值会不增反降；而对于深一些的网络，比如 YOLOX-L，在 Mosaic 基础上，增加 Mixup，AP 值会有所上升。因此不同的网络结构，采用数据增强的策略也不同，比如 YOLOX-s、YOLOX-m，或者 YOLOv4、YOLOv5 系列，都可以使用不同的数据增强策略进行尝试
- Scale 增强策略：在 Mosaic 数据增强中，`random_perspective` 函数，生成仿射变换矩阵时，对于图片的缩放系数，会生成一个随机值。对于 YOLOX-l 来说，随机范围 `scale` 建议设置在 $[0.1, 2]$ 之间，而对于轻量级模型，比如 YOLONano 时，随机范围 `scale` 建议设置在 $[0.5, 1.5]$ 之间，弱化 Mosaic 增广的性能。

4.6.2 YOLOX-Darknet53

YOLOX-d 的主干网络，和原本的 YOLOv3 baseline 的主干网络都是一样的。

4.6.3 Neck

在 Neck 结构中，YOLOX-Darknet53 和 YOLOv3 baseline 的 Neck 结构，也是一样的，都是采用 FPN^{4.4.3}的结构进行融合。

如图¹⁴所示，FPN 自顶向下，将高层的特征信息，通过上采样的方式进行传递融合，得到进行预测的特征图。而在 YOLOv4、YOLOv5、甚至后面讲到的 YOLOX-s、l 等版本中，都是采用 FPN+PAN 的形式

4.6.4 Prediction 层

在输出层中，主要从四个方面进行讲解：Decoupled Head、Anchor Free、标签分配、Loss 计算。

- Decoupled Head：目前在很多一阶段网络中都有类似应用，比如 RetinaNet、FCOS 等。在 YOLOX 中，作者增加了三个 Decoupled Head，俗称“解耦头”。

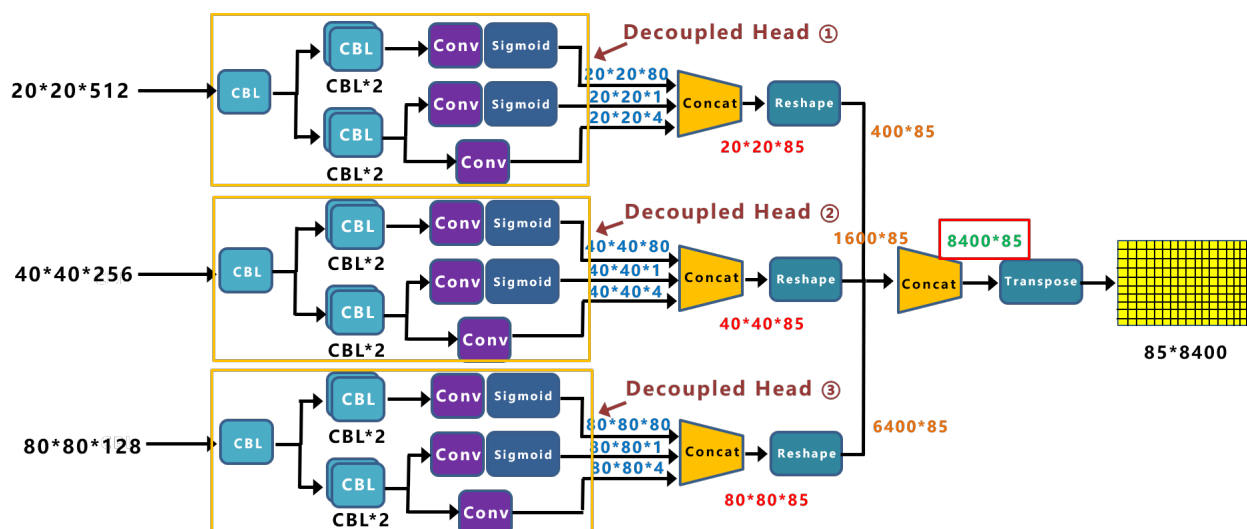


图 15: Decoupled_Head 结构图

提醒：为什么使用 Decoupled Head？

作者将输出端改进为 End-to-end 的方式（即无 NMS 的形式）。发现，改进完之后的 AP 值下降了 4.2 个百分点（38.5->34.3）。而将 End-to-End 中的 Yolo Head，修改为 Decoupled Head 的方式后，AP 值从 34.3 增加到了 38.8。然后作者又将 YOLOv3 baseline 中 Yolo Head，也修改为 Decoupled Head。发现 AP 值，从 38.5 增加到 39.6，并且训练收敛的速度也加快了。但是将检测头解耦，会增加运算的复杂度。

- Anchor free: 目前行业内，主要有 Anchor Based 和 Anchor Free 两种方式。在 YOLOv3、YOLOv4、YOLOv5 中，通常都是采用 Anchor Based 的方式，来提取目标框，进而和标注的 groundtruth 进行比对，判断两者的差距。

提醒：Anchor Based 和 Anchor Free 的区别

- Anchor Based: 设置 Anchor 规则，将预测框和标注框进行关联。基于每个单元格，都有三个不同尺寸大小的锚框。以输入图像大小 416×416 为例。网络最后的三个特征图大小为 13×13 , 26×26 , 52×52 。当采用 COCO 数据集，即有 80 个类别时。基于每个锚框，都有 x、y、w、h、obj（前景背景）、class（80 个类别），共 85 个参数。因此会产生 $3 \times (13 \times 13 + 26 \times 26 + 52 \times 52) \times 85 = 904995$ 个预测结果。如果将输入从 416×416 ，变为 640×640 ，最后的三个特征图大小为 20×20 , 40×40 , 80×80 。则会产生 $3 \times (20 \times 20 + 40 \times 40 + 80 \times 80) \times 85 = 2142000$ 个预测结果
- Anchor Free: 以输入图像大小 640×640 为例，15 中黄色的 85×8400 ，不是类似于 YOLOv3 中的 Feature Map，而是特征向量。比基于 Anchor Based 的方式，少了 2/3 的参数量。

提醒： Anchor Based 的局限性！

- Anchor 的设置需要手动设计
- Anchor 的匹配机制使得极端尺度被匹配到的频率较低
- Anchor 的庞大数量使得存在严重的不平衡问题
- 为了兼顾多尺度下的预测能力，推理得到的预测框也相对较多，nms 耗时较大

提醒： Anchor Free 就是真的没有锚框吗？

其实并不然，YOLOX-Darknet53 只是巧妙的将前面 Backbone 中，下采样的大小信息引入进来。图14最上面的分支，下采样了 5 次 ($2^5 = 32$)，输出大小为 $20 \times 20 \times 85$ 。也就是说**最后 8400 个预测框中，其中有 400 个框，所对应锚框的大小，为 32×32** 。同样的原理，中间的分支，最后有 1600 个预测框，所对应锚框的大小，为 16×16 。最下面的分支，最后有 6400 个预测框，所对应锚框的大小，为 8×8 。当有了 8400 个预测框的信息，每张图片也有标注的目标框的信息。这时需要做的，就是将 8400 个锚框，和图片上所有的目标框进行关联，挑选出正样本。而相应的，正样本锚框所对应的位置，就可以将正样本预测框，挑选出来。这里采用的关联方式，是**标签分配**。

简单地说，Anchor Base 指的是使用聚类方法生成多种宽高比的锚框，模型在预测 Bounding Box 时其实是在预测 Bound Box 的中心点和锚框中心点的偏移以及宽高比例。Anchor Free 中，模型是直接预测 Bound Box 的宽高值。

提醒： 如何实现标签分配？

需要明确一点，预测框只有少部分是正样本，绝大多数是负样本。那么如何挑选正样本呢？这里就涉及到两个关键点：初步筛选和 SimOTA。

- 初步筛选：初步筛选的方式主要有两种：根据中心点来判断、根据目标框来判断；
 - 根据中心点来判断：寻找 `anchor_box` 中心点，落在 `groundtruth_boxes` 矩形范围的所有 anchors
 - 根据目标框来判断：以 `groundtruth` 中心点为基准，设置边长为 5 的正方形，挑选在正方形内的所有锚框。
- 引入 SimOTA，实现精细化筛选：整个筛选流程主要分为 5 个阶段：
 - 初筛正样本：初筛出的 1000 个正样本锚框的位置
 - 计算 anchor 与 gt 的 iou
 - 在候选区域内计算 cost。开始 $L_{cls} : L_{reg} = 1 : 3$ ，而最终计算 loss 时， $L_{cls} : L_{reg} = 1 : 5$
猜测可能是在寻找正负样本时，如果一个 anchor 已经能很好的区分前景和背景了，那么即使 reg 预测不准，也可以作为正样本，所以相比较 loss 计算，在 cost 计算时，Lcls 给予了更大的权重。而 reg 可以通过不断地训练逐步优化
 - 获取当前 gt 的 iou 前 10（在 5-15 之间取值，结果的影响差距较小）的样本；并将这 top10 样本的 iou 求和取整，作为当前 gt 需要分配到的正样本数 `dynamic_k`（最小取 1，由于前期模型预测不准，导致 iou 基本比较小，所以观测到前期 `dynamic_k` 大多为 1，**会不会导致训练速度变缓的情况？**）
 - 舍去全局优化求解过程，只为每个 gt 取 cost 排名最小的前 `dynamic_k` 个 anchor 作为正样本，其余为负样本
 - 人工去掉同一个样本被分配到多个 gt 的正样本的情况

注：SimOTA 与 OTA 相比在最后两步做了简化，因此比 OTA 的运算速度更快，训练时间更短，不需要额外的优化参数步骤，同时保证了精度在绝大多数数据集下几乎没有影响。

4.7 YOLOV6

注：有争议，像大杂烩，[参考](#)

YOLOv6 主要在 Backbone、Neck、Head 以及训练策略等方面进行了诸多的改进：

- 统一设计了更高效的 Backbone 和 Neck：受到硬件感知神经网络设计思想的启发，基于 RepVGG style 设计了可重参数化、更高效的骨干网络 EfficientRep Backbone 和 Rep-PAN Neck
- 优化设计了更简洁有效的 Efficient Decoupled Head，在维持精度的同时，进一步降低了一般解耦头带来的额外延时开销
- 在训练策略上，采用 Anchor-free 无锚范式，同时辅以 SimOTA 标签分配策略以及 SIOU2.2.6 边界框回归损失来进一步提高检测精度


```

self.cv2 = Conv(c1, c_, 1, 1)
self.cv3 = nn.ModuleList([Conv(c_ if i == 0 else c2, c2, 3, 1) for i in range(n)])
self.cv4 = Conv(c_ * 2 + c2 * (len(ids) - 2), c3, 1, 1)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    x_1 = self.cv1(x)
    x_2 = self.cv2(x)

    x_all = [x_1, x_2]
    for i in range(len(self.cv3)):
        x_2 = self.cv3[i](x_2)
        x_all.append(x_2)
    out = self.cv4(torch.cat([x_all[_id] for _id in self.ids], dim=1))
    return out

```

4.8.2 TransitionBlock&MP

主干网络的下采样层使用的是 TransitionBlock 和 MP 结构，该结构的特点是同时使用池化层（步长为 2）和卷积层（步长为 2），实现特征层的 2 倍降维。

```

class MP(nn.Module):
    def __init__(self, k: int = 2):
        super(MP, self).__init__()
        self.m = nn.MaxPool2d(kernel_size=k, stride=k)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.m(x)

class TransitionBlock(nn.Module):
    def __init__(self, c1: int, c2: int):
        super(TransitionBlock, self).__init__()
        self.cv1 = Conv(c1, c2, 1, 1)
        self.cv2 = Conv(c1, c2, 1, 1)
        self.cv3 = Conv(c2, c2, 3, 2)
        self.mp = MP()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x_1 = self.mp(x)
        x_1 = self.cv1(x_1)

        x_2 = self.cv2(x)
        x_2 = self.cv3(x_2)

        return torch.cat([x_2, x_1], 1)

```

4.8.3 Neck

Neck 部分使用了 RepConv 结构，与 yolov6^{4.7}中的结构一样，预测阶段会将多个卷积核融合成一个 3×3 的卷积，从而提高推理的速度。延续 yolov5 使用路径聚合网络^{4.4.4}，实现上层、底层特征融合。

4.8.4 Head

yolov7 的 head 与 yolov5 是一样的，预测的公式也是一样的。预测的是 grid 的中心点，因此再图像坐标系下允许偏移量的范围为-0.5 到 1.5。

4.8.5 正负样本筛选

yolov5 的正样本是静态选择，yolov7 已经升级为动态选择。会先使用 yolov5 的方法选择候选样本，然后再此基础之上使用 SimOTA 算法4.6.4进一步筛选正样本。SimOTA 会根据现阶段模型训练程度选择正样本数目，具体表现为依据 iou 确定正样本数目，依据 $3 * iou_loss$ 和 cls_loss 作为最终筛选依据。这里 iou_loss 的系数与 cls_loss 的系数之比为 3:1，目的是让该阶段更多关注与候选的 anchor 是不是目标，而不需要过分关注具体的类别。yolov5 候选正样本 (最多 27 个)。

- 对每一个 GT 框，分别计算与 anchor 的宽与宽的比值、高与高的比值
- 取比值在 1/4-4 之间的作为候选项; 取比值最接近的当做候选项
- 分别计算 ground_truth 中心点与候选 anchor 左上角的偏移量，确定是在第几象限，进而进一步确定扩展出来的两个 anchor 是在上，下，左，右哪一个方向;
- 对于同一个 anchor 匹配多个 ground_truth 情况，取最大 iou 值的 ground_truth;

4.9 YOLOV8

注：本身的创新点不多，融合众多当前 SOTA 技术于一体，偏向工程实践

YOLOv8

RangeKing

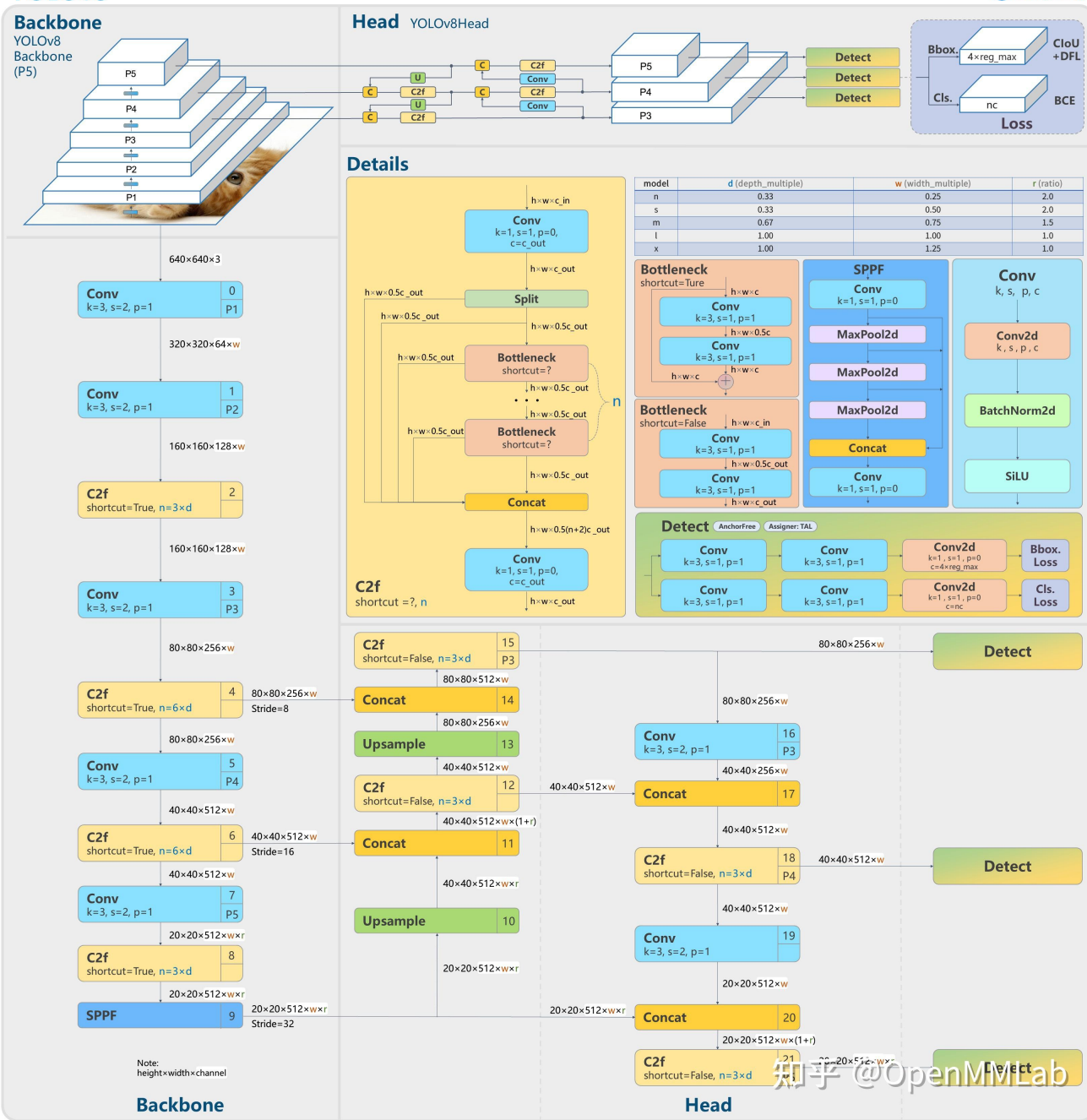


图 17: YOLOV8 结构图

按照官方描述, YOLOv8 是一个 SOTA 模型, 它建立在以前 YOLO 版本的成功基础上, 并引入了新的功能和改进, 以进一步提升性能和灵活性。具体创新包括: 新的骨干网络、新的 Anchor-Free 检测头和新的损失函数, 可以在从 CPU 到 GPU 的各种硬件平台上运行。

模型大小	YOLOV5 (300e)	params (M)	FLOPs@640 (B)	YOLOV8 (500e)	params (M)	FLOPs@640 (B)
n	28.0	1.9	4.5	37.3	3.2	8.7
s	37.4	7.2	16.5	44.9	11.2	28.6
m	45.4	21.2	49.0	50.2	25.9	78.9
l	49.0	46.5	109.1	52.9	43.7	165.2
x	50.7	86.7	205.7	53.9	68.2	257.8

4.9.1 Backbone & Neck

具体变化为：

- 第一个卷积层的 kernel 从 6×6 变成了 3×3 所有的 C3 模块换成 C2f，结构如下所示，可以发现多了更多的跳层连接和额外的 Split 操作（C2f 模块中存在 Split 等操作对特定硬件部署不太友好）

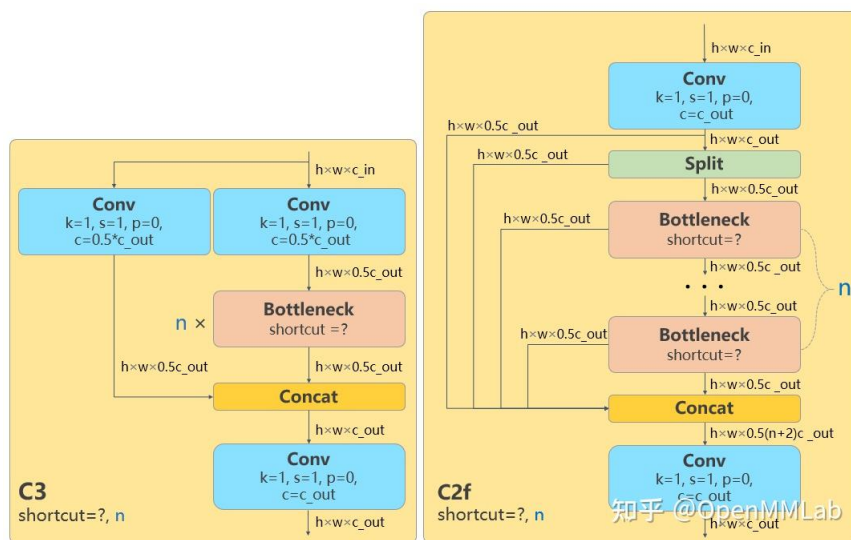


图 18: C2f 结构

- 去掉了 Neck 模块中的 2 个卷积连接层
- Backbone 中 C2f 的 block 数从 3-6-9-3 改成了 3-6-6-3
- N/S 和 L/X 两组模型只是改了缩放系数，但是 S/M/L 等骨干网络的通道数设置不一样，没有遵循同一套缩放系数。如此设计的原因应该是同一套缩放系数下的通道设置不是最优设计

4.9.2 Head

Head 部分从 YOLOv5 的 Anchor-Based 变成了 Anchor-Free，并且从耦合头变成了解耦头。其结构如下所示：

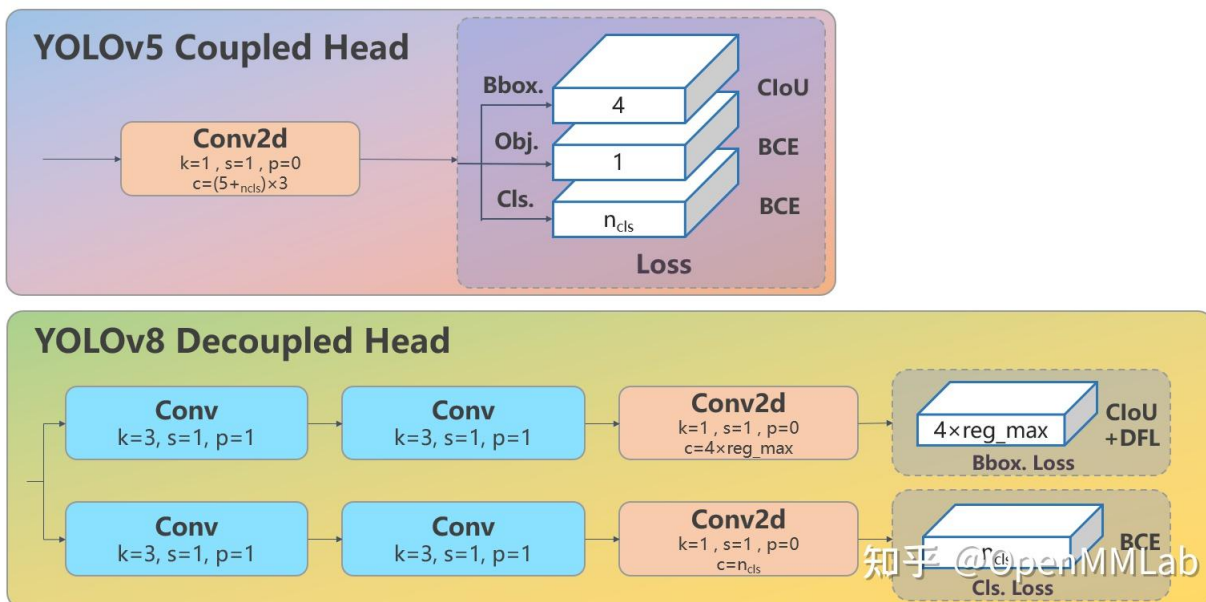


图 19: YOLOV8 的解耦头结构

4.9.3 TaskAlignedAssigner

现代目标检测器大部分都会在正负样本分配策略上面做文章，典型的如 YOLOX 的 simOTA、TOOD 的 TaskAlignedAssigner 和 RTMDet 的 DynamicSoftLabelAssigner，这类 Assigner 大都是动态分配策略 TaskAlignedAssigner 的匹配策略简单总结为：**根据分类与回归的分数加权的分数选择正样本。**

$$t = s^{\alpha} \times u^{\beta} \quad (42)$$

其中， s 是标注类别对应的预测分值， u 是预测框和 gt 框的 iou，两种相乘可以衡量对齐程度

- 对于每一个 GT，对所有的预测框基于 GT 类别对应分类分数，预测框与 GT 的 IoU 的加权得到一个关联分类以及回归的对齐分数 alignment_metrics
- 对于每一个 GT，直接基于 alignment_metrics 对齐分数选取 topK 大的作为正样本

提醒： 样本匹配策略总结

- 静态策略：基于经验得出，在训练开始之前就确定了，可以根据数据集的特点进行调整。为了让正样本分配更合理，提出了**正样本增样**。比如：
 - 跨分支采样：在三种尺寸的特征图上都对 GT 匹配
 - 跨 grid 采样：若 GT 落在某网格内，将此网格左、上、右、下 4 个领域内离 GT 中心较近的两个网格作为正样本
 - 跨 anchor 采样：根据宽高比匹配正样本。如果 anchor 和 GT 的宽高比小于阈值，就匹配成功。可理解为 GT 满足 anchor 宽和高都在 $\times 0.25$ 和 $\times 4.0$ 倍之间就算匹配成功
- 动态匹配4.9.3：可根据训练损失或其他指标来调整，可以更好的适应不同的数据集和模型

注：在 v5 中一个目标最多可有 9-27 个正样本

4.9.4 Loss 计算

Loss 不再有之前的 objectness 分支，只有解耦的分类和回归分支

- 分类分支：依然采用 BCE Loss
- 回归分支：需要和 DFL2.1.2中提出的积分形式表示法绑定，同时还使用了 CIoU2.2.4

4.10 YOLOV9

大清亡了！

4.10.1 创新点

解决深度神经网络中信息丢失所带来的挑战。信息瓶颈原理和可逆函数的创新使用是其设计的核心，可确保 YOLOv9 保持高效率和高精度。

4.10.2 信息瓶颈原理

信息瓶颈原理揭示了深度学习中的一个基本挑战：当数据通过网络的连续层时，信息丢失的可能性会增加。这一现象在数学上表现为

$$I(X, X) \geq I(X, f_{\theta}(X)) \geq I(X, g_{\phi}(f_{\theta}(X))) \quad (43)$$

其中 I 表示互信息，而 f 和 g 表示带参数的变换函数 θ 和 ϕ 分别为 YOLOv9 通过实施可编程梯度信息 (PGI) 应对了这一挑战，PGI 有助于保留整个网络深度的重要数据，确保更可靠的梯度生成，从而提高模型的收敛性和性能。

4.10.3 可逆函数

可逆函数的概念是 YOLOv9 设计的另一个基石。如果一个函数可以在不丢失任何信息的情况下被反转，那么这个函数就被认为是可逆的：

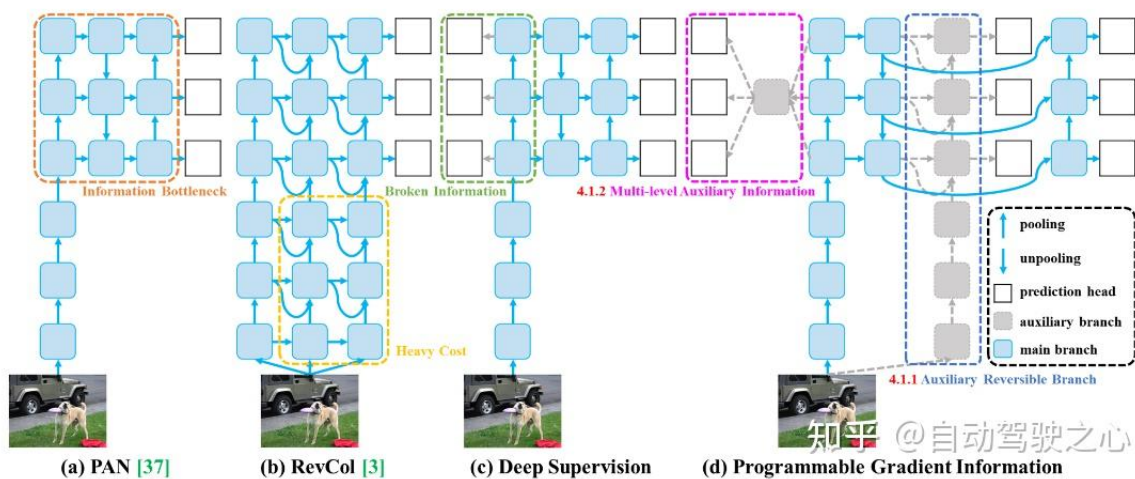
$$X = v_{\zeta}(r_{\psi}(X)) \quad (44)$$

与 ψ 和 ζ 分别作为可逆函数及其反函数的参数。这一特性对深度学习架构至关重要，因为它能让网络保留完整的信息流，从而更准确地更新模型参数。YOLOv9 在其架构中加入了可逆函数，以降低信息退化的风险，尤其是在较深的层中，从而确保为物体检测任务保留关键数据。

4.10.4 对轻型模型的影响

解决信息丢失问题对于轻量级模型尤为重要，因为轻量级模型通常参数化不足，在前馈过程中容易丢失大量信息。YOLOv9 的架构通过使用 PGI 和可逆函数，确保即使是精简的模型，也能保留并有效利用准确检测物体所需的基本信息。

4.10.5 可编程梯度信息 (PGI)



PGI 主要包括三个部分，

- 主分支：推理过程仅使用了主分支，因此不需要额外的推理成本；
- 辅助可逆分支：为了处理神经网络加深带来的问题，网络加深会造成信息瓶颈，导致损失函数无法生成可靠的梯度；
- 多级辅助信息：旨在处理深度监督带来的误差累积问题，特别是多个预测分支的架构和轻量级模型。

4.10.6 通用高效层聚合网络 (GELAN)

研究者把 CSPNet、ELAN^{4.8.1}这两种神经网络架构结合起来，从而设计出兼顾轻量级、推理速度和准确性的通用高效层聚合网络 (generalized efficient layer aggregation network, GELAN)，**增量式改进**。

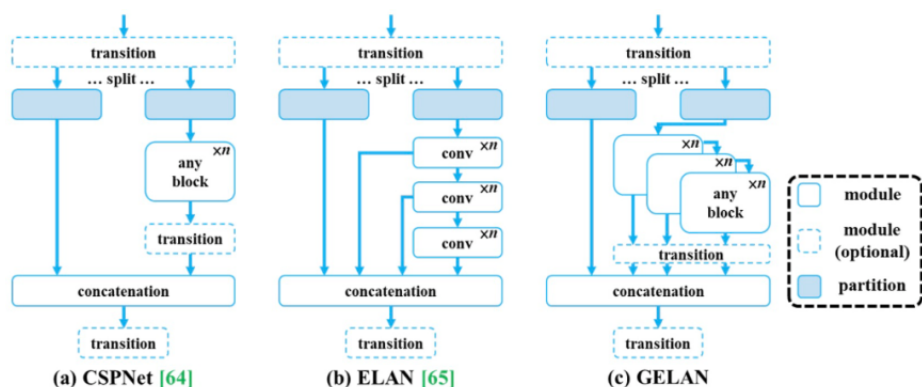


Figure 4. The architecture of GELAN: (a) CSPNet [64], (b) ELAN [65], and (c) proposed GELAN. We imitate CSPNet and extend ELAN into GELAN that can support any computational blocks.

5 Transformer 系列

5.1 DETR

目标检测变换为一个集合预测的问题，既不需要 proposal，也不需要 anchor，更不需要 nms

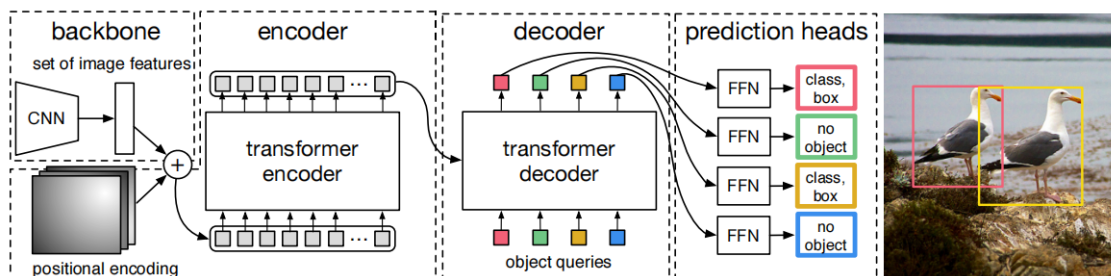


Fig.2: DETR uses a conventional CNN backbone to learn a 2D representation of an input image. The model flattens it and supplements it with a positional encoding before passing it into a transformer encoder. A transformer decoder then takes as input a small fixed number of learned positional embeddings, which we call *object queries*, and additionally attends to the encoder output. We pass each output embedding of the decoder to a shared feed forward network (FFN) that predicts either a detection (class and bounding box) or a “no object” class.

encoder-decoder 类似 unet，前者负责提取特征抽象化，后者负责区分目标边缘

```
class DETR(nn.Module):
    def __init__(self, num_classes, hidden_dim, nheads,
                 num_encoder_layers, num_decoder_layers):
        super().__init__()
        # We take only convolutional layers from ResNet-50 model
        self.backbone = nn.Sequential(*list(resnet50(pretrained=True).children())[:-2])
        self.conv = nn.Conv2d(2048, hidden_dim, 1)
        self.transformer = nn.Transformer(hidden_dim, nheads,
                                           num_encoder_layers, num_decoder_layers)
        self.linear_class = nn.Linear(hidden_dim, num_classes + 1)
```

```

self.linear_bbox = nn.Linear(hidden_dim, 4)
self.query_pos = nn.Parameter(torch.rand(100, hidden_dim))
self.row_embed = nn.Parameter(torch.rand(50, hidden_dim // 2))
self.col_embed = nn.Parameter(torch.rand(50, hidden_dim // 2))

def forward(self, inputs):
    x = self.backbone(inputs)
    h = self.conv(x)
    H, W = h.shape[-2:]
    pos = torch.cat([
        self.col_embed[:W].unsqueeze(0).repeat(H, 1, 1),
        self.row_embed[:H].unsqueeze(1).repeat(1, W, 1), 28 ], dim=-1).flatten(0, 1).unsqueeze(1)
    h = self.transformer(pos + h.flatten(2).permute(2, 0, 1),
        self.query_pos.unsqueeze(1))
    return self.linear_class(h), self.linear_bbox(h).sigmoid()

detr = DETR(num_classes=91, hidden_dim=256, nheads=8, num_encoder_layers=6, num_decoder_layers=6)
detr.eval()
inputs = torch.randn(1, 3, 800, 1200)
logits, bboxes = detr(inputs)

```

5.1.1 目标函数

原来是一对多，现在是强行限制一对一。

- 第一步完成最小代价匹配

Let us denote by y the ground truth set of objects, and $\hat{y} = \{\hat{y}_i\}_{i=1}^N$ the set of N predictions. Assuming N is larger than the number of objects in the image, we consider y also as a set of size N padded with $\emptyset$ (no object). To find a bipartite matching between these two sets we search for a permutation of N elements $\sigma \in \mathfrak{S}_N$ with the lowest cost:

$$\hat{\sigma} = \arg \min_{\sigma \in \mathfrak{S}_N} \sum_i^N \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}), \quad (1)$$

where $\mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$ is a pair-wise *matching cost* between ground truth y_i and a prediction with index $\sigma(i)$. This optimal assignment is computed efficiently with the Hungarian algorithm, following prior work (*e.g.* [43]).

- 第二步计算常规损失

pairs matched in the previous step. We define the loss similarly to the losses of common object detectors, *i.e.* a linear combination of a negative log-likelihood for class prediction and a box loss defined later:

$$\mathcal{L}_{\text{Hungarian}}(y, \hat{y}) = \sum_{i=1}^N \left[-\log \hat{p}_{\hat{\sigma}(i)}(c_i) + \mathbb{1}_{\{c_i \neq \emptyset\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\hat{\sigma}(i)}) \right], \quad (2)$$

where $\hat{\sigma}$ is the optimal assignment computed in the first step (1). In practice, we

为了模型收敛更快，会额外加一些辅助头

5.1.2 Object queries

类似于 anchor，anchor 是提前定义 banding box，Object queries 可学习

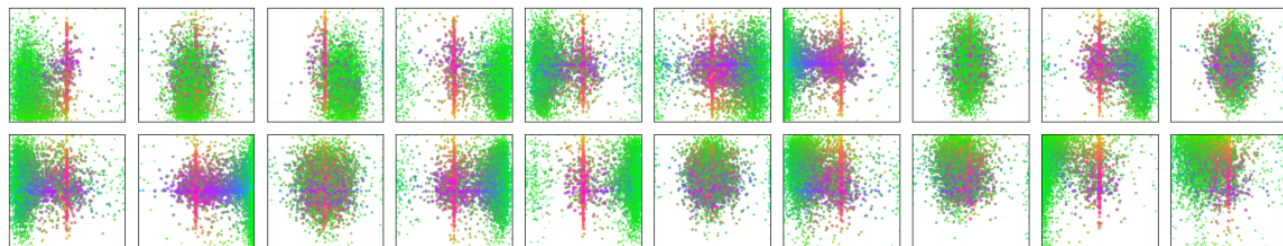


图 20: 可视化 100 个 Object queries 中的前 20 个。每个 box 被表示为一个点。绿色对应小 box，红色对应横向的大 box，蓝色对应竖向的大 box

基本上每个可视化图中间都有红色一条点，这个可能和 COCO 数据集有关，中间都会存在一些比较大的目标

5.1.3 DETR 的局限性

主要有：

- 训练时间长
- DETR 对大目标比较敏感，在小目标上的效果不太好

6 半监督学习

6.1 Softer Teacher[2]

FixMatch 提出了一个经典的数据增广方式来做 SSL：未标注的数据使用弱数据增强图像打伪标签，增强图像训练网络。

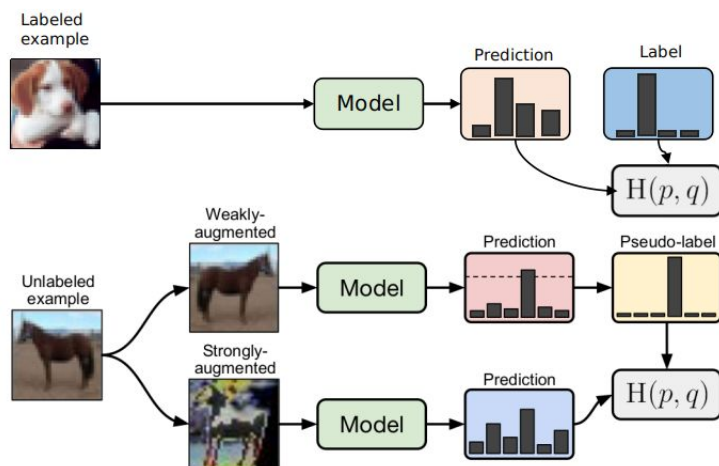


图 21: 经典的半监督学习方式

受 FixMatch 启发, STAC:

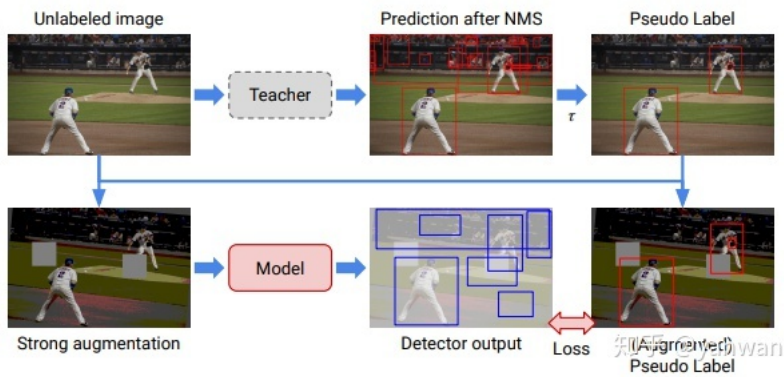


图 22: Caption

Softer Teacher 是一种端到端的半监督目标检测方法:

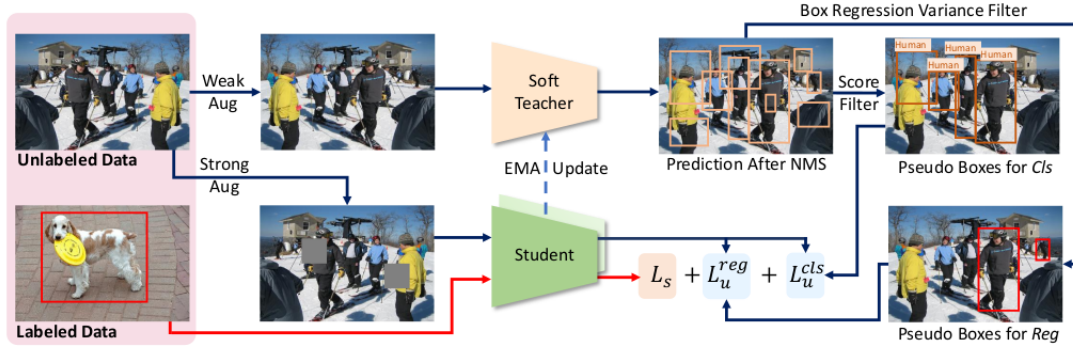


Figure 2. The overview of the end-to-end pseudo-labeling framework for semi-supervised object detection. Unlabeled images and labeled images form the training data batch. In each training iteration, a soft teacher is applied to perform pseudo-labeling on weak augmented unlabeled images on the fly. Two sets of pseudo boxes are produced: one is used for classification branch by filtering boxes according to the foreground score, and the other is used for box regression branch by filtering boxes according to box regression variance. The teacher model is updated by student model via exponential mean average (EMA) manner. The final loss is the sum of supervised detection loss L_s and unsupervised detection loss L_u .
CSDN @chenzy_hust

图 23: Caption

提醒： 现有半监督学习方法的缺点：初始检测器的性能影响伪标签的质量，会影响最终的性能；本文的两个创新点：

- 未标注边界框的分类损失由 teacher 网络产生；
- 框抖动方法【以框回归的方差作为定位可靠性度量】，用于选择可靠的伪框

损失函数由监督损失和无标注数据的无监督损失组成：

$$L = L_s + \alpha L_u \quad (45)$$

为了解决较高阈值导致的前景框被错分为背景类别的问题，Softer teacher 通过对所有背景样本评估其真实背景的可靠性，让可靠性低的背景样本损失更低：

$$L_u^{cls} = \frac{1}{N_b^{fg}} \sum_{i=1}^{N_b^{fg}} l_{cls}(b_i^{fg}, G_{cls}) + \sum_{j=1}^{N_b^{bg}} w_j l_{cls}(b_j^{bg}, G_{cls}), \quad w_j = \frac{r_j}{\sum_{k=1}^{N_b^{bg}} r_k} \quad (46)$$

一般地，通过 teacher 模型的预测得分就可以确定伪候选框的类别和坐标。事实上，前景得分区分正负样本与最后的回归准确度并无正向关系。因此本文通过度量回归预测的一致性来估计伪候选框的定位可靠性。具体地，将 teacher 模型预测的前景得分大于 0.5 的框多次抖动，然后继续送入 teacher 模型进行回归，计算抖动框的回归方差，方差越小，定位可靠性越高。

$$\bar{\sigma}_i = \frac{1}{4} \sum_{k=1}^4 \hat{\sigma}_k, \quad \hat{\sigma}_k = \frac{\sigma_k}{0.5(h(b_i) + w(b_i))} \quad (47)$$

参考文献

- [1] S. Ren, K. He, R. Girshick & J. Sun, Faster R-CNN: Towards real-time object detection with region proposal networks, *Computer Vision and Pattern Recognition*, 2015.
- [2] M. Xu, Z. zhang, H., Hu, et al. End-to-End semi-supervised object detection with soft teacher, *Computer Vision and Pattern Recognition*, 2021.